



Práctica 2. Sockets

Prof Carmen Lucia Bustillo Hernández
carmenbustillohernandez@gmail.com



Facultad de Ingeniería
Asignatura: Sistemas Distribuidos
Universidad Nacional Autónoma de México
Fecha de Entrega: 9 de Abril de 2026

Datos del Alumno

Nombre: Enrique Yoab Cortez Rios
Matrícula: 317302992

Objetivo General

El alumno comprenderá el funcionamiento de los diferentes algoritmo de sincronización para sistemas distribuidos e implementará los mismos mediante simulaciones utilizando procesos e hilos respectivamente.

Objetivos Específicos

1. Implementar la simulación de algoritmos de sincronización en sistemas distribuidos.

Material

1. Equipo de cómputo con sistema operativo Windows. Mac o Linux y acceso a Internet.
2. Editor de textos (MS Word, Libre Office).
3. Compilador o IDE de lenguajes de programación.

Bibliografía

1. Tanenbaum, A. S. (2004). Sistemas operativos modernos. En Prentice-Hall, Inc eBooks .
<http://dl.acm.org/citation.cfm?id=120575>

1. Instrucciones Generales

1. Realizar cada uno de los problemas prácticos con las indicaciones que se solicitan.
2. Para cada uno de los problemas, desarrolle su diagrama de flujo y pseudocódigo respectivamente.

2. Criterios de Evaluación

1. Se evaluará de acuerdo con los Criterios de Evaluación que se encuentra en el Google Classroom de la asignatura.

3. Problemas

3.1 Problema 1: Algoritmo de Sincronización de Lamport.

El objetivo de esta implementación es garantizar el ordenamiento causal de eventos en un sistema distribuido mediante el uso de **relojes lógicos escalares**.

3.1.1 Lógica de Actualización del Reloj.

En el archivo `cnx.lamport.c`, se implementan las reglas fundamentales de Lamport para ajustar el tiempo lógico en cada nodo:

- **Evento de Envío:** Antes de transmitir un mensaje, el proceso incrementa su reloj local.
- **Evento de Recepción:** Al recibir un mensaje externo, el nodo ajusta su reloj utilizando el valor máximo entre su tiempo actual y el tiempo recibido (T_s), sumando una unidad para representar el evento de recepción:

$$L(e) = \max(\text{reloj_local}, \text{reloj_externo}) + 1$$

Por otro lado, no podemos ignorar que el propio proceso genere eventos dentro de si mismo por lo que puede incrementar su reloj, ya se temporizadores, revisiones, protocolos, etc.

3.1.2 Arquitectura del Sistema.

La solución utiliza una arquitectura **descentralizada** y **multihilo**:

1. **Hilo de Escucha (`hilo_escucha`):** Actúa como servidor permanente utilizando sockets (TCP) para recibir estructuras de tipo `Mensaje_L`.
2. **Hilo de Envío:** Gestiona la salida de mensajes hacia otros nodos de la red de forma aleatoria, simulando la interacción entre procesos independientes.
3. **Proceso Gestor (Hijo):** Se encarga de generar `N_NODOS` que son los clientes. (Procesos Nietos)

4. **Proceso Maestro (Padre):** Coordina la simulación inyectando eventos de control (comando ENVIAR o LOCAL) sin alterar el tiempo lógico global, ya que actúa como un observador externo (ID 99).

3.1.3 Estructura de Datos.

Para la comunicación, se definió la estructura `Mensaje_L` en el archivo de cabecera `lamport_p2p.h`:

```
1 typedef struct {
2     int id_proceso;    // Id del proceso que envia
3     int indice;        // Indice en la lista de puertos
4     int reloj;         // Reloj logico escalar
5     char peticion[200]; // Mensaje o comando
6 } Mensaje_L;
```

3.1.4 Archivo Principal. (`sinc_lamport.c`)

Para comprender a detalle el funcionamiento de la simulación, debemos iniciar con el análisis del archivo principal, el cual coordina la ejecución global del sistema.

Este archivo incluye las librerías estándar y los archivos de cabecera necesarios para su operación. Define dos variables globales fundamentales: una estructura de tipo `Mensaje_L`, utilizada para inyectar instrucciones a los nodos de manera que interactúen entre sí sin modificar sus relojes lógicos (permitiendo que el nodo interprete el comando como una decisión “propia”), y un arreglo de enteros cuyo tamaño corresponde a la cantidad de nodos a crear, el cual permite gestionar los números de puerto que se asignarán a cada proceso.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <sys/wait.h>
6 #include <time.h>
7 #include "lamport_p2p.h"
8
9 Mensaje_L instancia;
10 int directorio[NUM_NODOS];
```

La primera sección del condicional `fork()` gestiona la lógica del proceso hijo, cuya responsabilidad es engendrar los n procesos nietos que conforman los nodos de la red. Cada uno de estos procesos nietos se inicializa con atributos específicos: un identificador único (ID), su índice correspondiente dentro del arreglo de puertos, el reloj lógico inicializado en cero y un buffer de peticiones limpio. Asimismo, se hereda el directorio global a su espacio de memoria, lo que garantiza que cada nodo pueda identificar y comunicarse con los demás elementos de la red una vez establecidos.

```
1 int main()
2 {
3     // Variable para guardar la ejecucion de fork y saber si es correcto.
4     pid_t pid = fork();
5     if(pid < 0) exit(1);
6
7     // Proceso hijo creara los n nodos clientes con conexion peer to peer
8     if(pid == 0)
9     {
10        // Llenamos el directorio que almacena los puertos de cada nodo
11        for(int i = 0; i < NUM_NODOS; i++) directorio[i] = 3000 + i;
12
13        // For para crear N nodos cada quien con su respectivo puerto
14        for(int i = 0; i < NUM_NODOS; i++)
15        {
16            // El proceso nieta rellena el mensaje que sera almacenado en la maquina p2p
17            if(fork() == 0)
18            {
19                instancia.id_proceso = 5 * (i + 1);
20                instancia.indice = i;
21                instancia.reloj = 0;
22                memset(instancia.peticion, 0, sizeof(instancia.peticion));
23
24                // Se invoca la funcion de red con la configuracion local y el
25                directorio
26                conexion_p2p(instancia, directorio);
27                exit(0);
28            }
29            // Espera sincronizada de los procesos nietos
30            for(int i = 0; i < NUM_NODOS; i++) wait(NULL);
31            exit(0);
32        }
33    }
```

El proceso padre actúa como el controlador maestro de la simulación, ejerciendo su voluntad para inyectar eventos de manera aleatoria en los nodos de la red. Mientras que los nodos permanecen en un estado de escucha activa para recibir, interpretar y procesar datagramas —actualizando su reloj lógico y retornando a la inactividad según sea necesario—, el padre dinamiza el sistema enviando instrucciones externas.

Para lograr esto, se utiliza una estructura `Mensaje_L` identificada con el ID 99; este valor permite que los nodos receptores reconozcan al proceso padre y ejecuten la acción solicitada sin alterar su propio reloj lógico. A través de un ciclo `for`, el sistema inyecta n eventos aleatorios (locales o de envío), emulando la

interacción real de un sistema distribuido. Por otra parte es fundamental destacar que, antes de iniciar la interacción, el proceso padre debe esperar un tiempo de estabilización (en este caso, 1 segundo para 3 nodos). Este margen asegura que todos los nodos P2P hayan completado su inicialización y estén listos para aceptar conexiones. Cabe mencionar que, si se incrementara el número de nodos, este tiempo de espera debería aumentarse proporcionalmente para garantizar la integridad de la red. Finalmente, una vez cumplida la cuota de eventos, el padre ejecuta un protocolo de apagado global mediante el comando EXIT, garantizando que todos los procesos finalicen correctamente y evitando la permanencia de procesos “zombies” en el sistema operativo.

```
1 // El padre inyecta comandos de evento y apagado a los nodos
2 else
3 {
4     srand(time(NULL) ^ getpid() << 16);
5     sleep(1); // Tiempo de espera para estabilizacion de la red
6
7     printf("\n===== \n");
8     printf(" [PADRE] INICIANDO SIMULACION P2P (RELOJES DE LAMPORT) \n");
9     printf("===== \n");
10
11     Mensaje_L inyeccion;
12     for(int i = 0; i < NUM_EVENTO; i++)
13     {
14         int indice_elegido = rand() % NUM_NODOS;
15         int puerto_elegido = 3000 + indice_elegido;
16         memset(&inyeccion, 0, sizeof(Mensaje_L));
17
18         inyeccion.id_proceso = 99; // ID de control del padre
19         inyeccion.reloj = 0;      // El padre no altera el tiempo logico
20
21         // Seleccion aleatoria del tipo de evento
22         if(rand() % 2 == 1) strcpy(inyeccion.peticion, CMD_EN);
23         else strcpy(inyeccion.peticion, CMD_IN);
24
25         enviar_msj(puerto_elegido, inyeccion);
26         usleep(100000);
27     }
28
29     printf("\n [PADRE] INICIANDO PROTOCOLO DE APAGADO GLOBAL \n");
30     for(int i = 0; i < NUM_NODOS; i++)
31     {
32         memset(&inyeccion, 0, sizeof(Mensaje_L));
33         inyeccion.id_proceso = 99;
34         strcpy(inyeccion.peticion, CMD_EX);
35         enviar_msj(3000 + i, inyeccion);
```

```
36         usleep(50000);
37     }
38
39     wait(NULL);
40     printf("\n[SISTEMA GLOBAL] Nodos finalizados exitosamente.\n");
41 }
42 return 0;
43 }
```

- Archivo de Cabecera (lamport_p2p.h)

Este archivo de cabecera es fundamental para la cohesión del sistema, ya que define la estructura de datos y las constantes globales necesarias para el funcionamiento de la red. Como se puede observar, incluye las declaraciones de funciones que no están implementadas en el archivo principal; esto se debe a que la lógica de la conexión P2P y el protocolo de envío de mensajes se encuentran encapsulados en un módulo independiente. Esta modularización permite la reutilización de código y mantiene una separación clara entre la gestión de procesos y la comunicación por sockets.

```
1  #ifndef LAMPORT_P2P_H
2  #define LAMPORT_P2P_H
3
4  #define NUM_NODOS 3      // Numero de computadoras a crear
5  #define NUM_EVENTO 7    // Numero de eventos totales
6  #define CMD_EX "EXIT"   // Comando de apagado
7  #define CMD_EN "ENVIAR" // Comando para que la computadora mande un mensaje
8  #define CMD_IN "LOCAL"  // Comando de para que realice un evento interno
9  #define DIRECTION "127.0.0.1" // Direccion ip a la que se comunica
10
11 // ----- Estructura del mensaje de Lamport -----
12 typedef struct {
13     int id_proceso;      // Id del proceso que envia el mensaje
14     int indice;         // El indice al que corresponde
15     int reloj;          // El Reloj de Lamport escalar (un solo contador)
16     char peticion[200];  // Texto del mensaje a enviar
17 } Mensaje_L;
18
19 // ----- Declaraciones -----
20 int mayor(int a, int b);
21 void conexion_p2p(Mensaje_L user, int dir[]);
22 void enviar_msj(int puerto_destino, Mensaje_L datos);
23 void *hilo_escucha(void *arg);
24
25 #endif
```

3.1.5 Archivo de Comunicación P2P. (cnx_lamport.c)

La primera sección de la implementación del nodo P2P incluye las bibliotecas necesarias para la gestión de sockets, el manejo de hilos y la generación de números aleatorios. Un aspecto fundamental de este diseño es que, al ser invocado tras un `fork()`, cada proceso cuenta con su propio espacio de memoria aislado. Esto permite el uso de variables globales que cada nodo interpreta como su estado interno privado.

Las variables globales que definen el comportamiento de cada nodo son:

- **PORT:** Almacena el número de puerto asignado al nodo para la escucha de conexiones.
- **local:** Estructura que mantiene el ID, índice y reloj lógico del nodo, además del buffer de peticiones para la transmisión de datos.
- **externo:** Estructura destinada a la recepción y procesamiento de mensajes provenientes de otros nodos o del proceso padre.
- **puertos[]:** Arreglo que contiene la totalidad de los puertos de la red, permitiendo la comunicación aleatoria durante un evento de envío.
- **encendido:** Bandera de control que mantiene al nodo activo en modo de escucha y envío; su desactivación provoca el cierre de los sockets.
- **enviar:** Indicador que permanece en estado inactivo (0) y se activa únicamente al recibir una solicitud para iniciar una transmisión hacia otro nodo.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <arpa/inet.h>
6 #include <pthread.h>
7 #include <time.h>
8 #include "lamport_p2p.h"
9
10 // Al ser memoria aislada por fork, cada nodo posee sus propias variables
11 int PORT = 0;           // Puerto de escucha del nodo
12 Mensaje_L local;       // Estado y buffer de envio local
13 Mensaje_L externo;     // Buffer de recepcion externo
14 int puertos[NUM_NODOS]; // Directorio de puertos de la red
15 int encendido = 1;     // Control de vida del proceso
16 int enviar = 0;        // Flag de activacion para envio
```

Las siguientes dos funciones determinan el comportamiento central y las reglas matemáticas del reloj lógico:

- **mayor:** Devuelve el valor máximo entre dos números enteros mediante un operador ternario. Aunque es una función muy simple, es la pieza matemática clave para aplicar la regla de Lamport durante la recepción de un mensaje ($\max(\text{reloj_local}, \text{reloj_externo})$).
- **conexion_p2p:** Es la función principal que orquesta el comportamiento dual del nodo en la red. Primero, inicializa su estado interno: hereda el directorio de puertos, asigna su puerto de escucha local, registra su identificador e incrementa su reloj lógico (representando su primer evento interno de inicialización).

Posteriormente, emplea la biblioteca `pthread` para delegar a un hilo secundario la tarea de escuchar mensajes entrantes (`hilo_escucha`). Mientras tanto, el hilo principal entra en un bucle activo (`while(encendido)`) actuando como cliente. Dentro de este bucle, el nodo monitorea la bandera `enviar`; si esta se activa, el nodo aplica la **regla de envío de Lamport** (incrementando su reloj), selecciona de forma aleatoria un nodo destino (garantizando no enviarse un mensaje a sí mismo), construye la carga útil del mensaje y efectúa la transmisión. Finalmente, cuando el sistema ordena el apagado, utiliza `pthread_join` para asegurar que el hilo servidor termine su ejecución de manera limpia.

```
1 // Esta funcion encuentra el mayor de dos numeros con el operador ternario
2 int mayor(int a, int b){return a > b ? a : b;}
3
4 void conexion_p2p(Mensaje_L user, int dir[])
5 {
6     // Creamos una variable tipo hilo para la creacion del hilo servidor
7     pthread_t thread_id;
8     // Registramos el directorio de puertos
9     memcpy(&puertos, dir, sizeof(puertos));
10    // Registramos el puerto que desea utilizar y escuchar
11    PORT = dir[user.indice]; // Usamos el indice pasado como argumento para saber
    nuestro puerto
12    // Registramos el id que se le asigna
13    local.id_proceso = user.id_proceso;
14    // Registramos el indice al que corresponde su puerto
15    local.indice = user.indice;
16    // Inicializacion del reloj logico de Lamport
17    local.reloj++;
18
19    // Se crea un hilo que actuara como servidor (recibe mensajes)
20    pthread_create(&thread_id, NULL, hilo_escucha, NULL);
21
22    srand(time(NULL) ^ getpid()); // Semilla unica segura
23
24    // Actua como otro hilo que se encarga de ser cliente (envia mensajes)
25    while (encendido)
```

```
26 {
27     // Si enviar cambia se realiza la accion de enviar mensaje
28     if (enviar == 1)
29     {
30         // Invertimos la condicion ya que es solo una vez
31         enviar = 0;
32
33         // REGLA DE LAMPORT AL ENVIAR: Avanzar reloj antes de enviar
34         local.reloj++;
35
36         int indice_new;
37         // Obtenemos un indice aleatorio y evitamos enviarnos a nosotros mismos si
no repite
38         do{
39             indice_new = rand() % NUM_NODOS;
40         } while (indice_new == local.indice);
41
42         printf("\n[C%02d] Enviare mensaje a la computadora con puerto %d...\n",
local.id_proceso, puertos[indice_new]);
43         // Limpiamos nuestro buffer de peticiones
44         memset(local.peticion, 0, sizeof(local.peticion));
45         // Le grabamos el mensaje al buffer de peticiones
46         snprintf(local.peticion, sizeof(local.peticion), "Soy la computadora %02d",
local.id_proceso);
47         // Enviamos el mensaje al puerto aleatorio escogido.
48         enviar_msj(puertos[indice_new], local);
49     }
50 }
51 // Esperamos a que termine la ejecucion del hilo servidor
52 pthread_join(thread_id, NULL);
53 }
```

Las últimas dos funciones constituyen el núcleo operativo de la simulación, permitiendo la coexistencia de los roles de servidor y cliente en cada proceso:

- **hilo_escucha:** Esta función se ejecuta de forma asíncrona y actúa como el servicio de recepción del nodo. Configura un socket de tipo `SOCK_STREAM` (TCP) e implementa la opción `SO_REUSEADDR` para garantizar que el puerto pueda ser reutilizado inmediatamente tras un reinicio, evitando estados de espera del sistema operativo. Una vez vinculado al puerto correspondiente (`bind`) y puesto en modo de escucha (`listen`), el hilo entra en un ciclo infinito de procesamiento.

Al aceptar una conexión, extrae la estructura `Mensaje_L` y discrimina su origen: si el emisor posee el **ID 99** (el proceso padre), interpreta la carga útil como una instrucción de control (`ENVIAR`, `LOCAL` o `EXIT`). Si el mensaje proviene de otro nodo de la red, se aplica estrictamente la **segunda regla de**

Lamport: el reloj local se actualiza tomando el valor máximo entre el contador propio y el recibido, sumando una unidad para representar el evento de recepción ($L(e) = \max(L_{local}, L_{externo}) + 1$).

- **enviar_msj:** Es una función utilitaria que abstrae la lógica de cliente. Se encarga de crear un socket temporal, establecer una conexión directa con el puerto del nodo destino y transmitir la estructura de datos completa. El uso de TCP garantiza que el mensaje llegue íntegro, permitiendo que el nodo emisor cierre el socket inmediatamente después de la transmisión para liberar recursos del sistema.

```
1 // Funcion que utilizara el hilo para actuar como servidor
2 void *hilo_escucha(void *arg)
3 {
4     int server_fd;    // Descriptor del socket servidor (el que escucha conexiones
                        // entrantes)
5     int new_socket;   // Descriptor del socket aceptado (uno por cada cliente que se
                        // conecta)
6     struct sockaddr_in address; // Estructura que almacena la dirección IP y puerto del
                        // servidor
7     int opt = 1;      // Valor para activar la opción SO_REUSEADDR en el socket
8
9     // Creamos el socket TCP (SOCK_STREAM) que actuará como servidor de este nodo
10    server_fd = socket(AF_INET, SOCK_STREAM, 0);
11
12    // Permite reutilizar el puerto inmediatamente después de que el proceso termine,
13    // evitando el error "Address already in use" en reinicios rápidos
14    setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
15
16    address.sin_family      = AF_INET;        // Familia de direcciones IPv4
17    address.sin_addr.s_addr = INADDR_ANY;     // Acepta conexiones de cualquier IP local
18    address.sin_port        = htons(PORT);    // Convierte el puerto a orden de bytes de
                        // red (big-endian)
19
20    // Asociamos el socket al puerto y dirección configurados en 'address'
21    bind(server_fd, (struct sockaddr *)&address, sizeof(address));
22
23    // Se pone el socket en modo escucha; admite hasta 3 conexiones en cola de espera
24    listen(server_fd, 3);
25
26    printf("[C%02d] Escuchando Mensaje_L en puerto %d...\n", local.id_proceso, PORT);
27
28    // Bucle activo mientras la bandera 'encendido' sea 1
29    while (encendido)
30    {
```

```
31     int addrlen = sizeof(address);    // Tamaño de la estructura de dirección (
requerido por accept)
32
33     // Espera bloqueante: se desbloquea cuando llega una conexión entrante.
34     // Devuelve un nuevo descriptor (new_socket) exclusivo para esa conexión.
35     new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t *)&
addrlen);
36
37     // Si accept falla (por ejemplo, el socket fue cerrado externamente) salimos del
bucle
38     if (new_socket < 0) break;
39
40     // Limpiamos el buffer 'externo' para evitar basura de mensajes anteriores
41     memset(&externo, 0, sizeof(Mensaje_L));
42
43     // Leemos del socket el bloque de bytes que forma el Mensaje_L completo
44     read(new_socket, &externo, sizeof(Mensaje_L));
45
46     // Verificamos el tipo de petición recibida para actuar en consecuencia
47     if(externo.id_proceso == 99)
48     {
49         if (strcmp(externo.peticion, CMD_EX) == 0)
50         {
51             // Comando de apagado: se baja la bandera para que ambos hilos terminen
52             printf("\n[C%02d] Apagando por comando remoto...\n", local.id_proceso);
53             encendido = 0;    // El hilo cliente y este hilo servidor saldrán de sus
bucles
54         }
55         else if (strcmp(externo.peticion, CMD_EN) == 0)
56         {
57             // Comando de envío: activa la bandera para que el hilo cliente envíe un
mensaje
58             enviar = 1; // El hilo cliente detectará esto en su próxima iteración
59         }
60         else if (strcmp(externo.peticion, CMD_IN) == 0)
61         {
62             printf("\n[C%02d] -> EVENTO INTERNO <- Ejecutando tarea local...\n",
local.id_proceso);
63             local.reloj++;
64         }
65     }
66     else
67     {
```

```
68         // REGLA DE LAMPORT AL RECIBIR
69         // Si el emisor es el proceso padre (id == 99), es un comando de control,
70         // no un evento de la red P2P real, por lo que NO se actualiza el reloj.
71         // Si es un nodo real, aplicamos: reloj_local = max(reloj_local,
reloj_externo) + 1
72         // El +1 representa el evento de recepción del mensaje.
73         local.reloj = mayor(local.reloj, externo.reloj) + 1;
74     }
75
76     // Imprimimos el detalle del mensaje recibido y el estado del reloj
77     printf("\nInicio -> Mensaje Computadora [C%02d] <-\n", local.id_proceso);
78     printf("---- ID_proceso: %d\n",                externo.id_proceso); // Quién envió
el mensaje
79     printf("---- Peticion: %s\n",                externo.peticion); // Contenido
del mensaje
80     printf("---- Reloj Externo (Ts): %d\n",        externo.reloj); // Reloj del
emisor
81     printf("---- Reloj Local Actualizado: %d\n", local.reloj); // Reloj local
tras el merge
82     printf("Fin -> Mensaje Computadora [C%02d] <-\n", local.id_proceso);
83
84     // Cerramos el socket de esta conexión específica; el servidor queda listo
85     // para aceptar la siguiente conexión en la próxima iteración del while
86     close(new_socket);
87 }
88
89 // Cerramos el socket servidor al salir del bucle (nodo apagado)
90 close(server_fd);
91 return NULL; // El hilo termina y libera sus recursos
92 }
93
94 // Esta funcion se encargara de enviar un mensaje a un puerto en especifico
95 void enviar_msj(int puerto_destino, Mensaje_L datos) {
96     int sock = 0;
97     struct sockaddr_in serv_addr;
98     // Creacion del socket tipo TCP para garantizar el envio y recepcion de mensaje
99     if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) return;
100
101     serv_addr.sin_port = htons(puerto_destino);
102     serv_addr.sin_family = AF_INET;
103     inet_pton(AF_INET, DIRECTION, &serv_addr.sin_addr);
104
105     // Tratamos de conectarnos si no puede nos salimos del programa
```

```
106     if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
107         perror("Error de conexion");
108         close(sock);
109         return;
110     }
111     // Enviamos el mensaje al conectarnos en el puerto definido
112     send(sock, &datos, sizeof(Mensaje_L), 0);
113     // Cerramos el socket ya que enviamos.
114     close(sock);
115 }
```

3.1.6 Pseudocódigo.

Pseudocódigo: sinc.lamport.c (P2P)

Algoritmo Sincronizacion_Lamport

Constantes y Estructuras (lamport_p2p.h)

- 1: Definir $NUM_NODOS \leftarrow 3$
- 2: Definir $NUM_EVENTO \leftarrow 7$
- 3: Definir $CMD_EX \leftarrow "EXIT"$
- 4: Definir $CMD_EN \leftarrow "ENVIAR"$
- 5: Definir $CMD_IN \leftarrow "LOCAL"$
- 6: Definir $DIRECTION \leftarrow "127.0.0.1"$
- 7: **Estructura** *Mensaje_L*
- 8: $id_proceso \in \mathbb{Z}$
- 9: $indice \in \mathbb{Z}$
- 10: $reloj \in \mathbb{Z}$
- 11: $peticion \in \text{Alfabeto}[200]$
- 12: **FinEstructura**

Variables Globales

- 1: Definir $instancia \in \text{Mensaje_L}$
- 2: Definir $directorio \in \mathbb{Z}[NUM_NODOS]$

1: **Procedimiento** MAIN

- 2: $pid \leftarrow fork()$
- 3: **Si** $pid < 0$ **Entonces**
- 4: **Salir**(1)
- 5: **Fin Si**
- 6: **Si** $pid = 0$ **Entonces** ▷ Proceso hijo creador de nodos P2P
- 7: **Para** $i \leftarrow 0$ **hasta** $NUM_NODOS - 1$ **Hacer**

```

8:      directorio[i] ← 3000 + i
9:      Fin Para
10:     Para i ← 0 hasta NUM_NODOS − 1 Hacer
11:         Si fork() = 0 Entonces                                ▷ Proceso nieto (Nodo P2P individual)
12:             instancia.id_proceso ← 5 * (i + 1)
13:             instancia.indice ← i
14:             instancia.reloj ← 0
15:             LimpiarMemoria(instancia.peticion)
16:             conexion_p2p(instancia, directorio)
17:             Salir(0)
18:         Fin Si
19:     Fin Para
20:     Para i ← 0 hasta NUM_NODOS − 1 Hacer
21:         EsperarHijo()
22:     Fin Para
23:     Salir(0)
24: Sino                                ▷ Proceso padre inyector de comandos
25:     GenerarSemilla(TiempoActual() XOR (ObtenerPID() << 16))
26:     DormirSegundos(1)
27:     Escribir("=====")
28:     Escribir(" [PADRE] INICIANDO SIMULACIÓN P2P (RELOJES DE LAMPORT) ")
29:     Escribir("=====")
30:     Definir inyeccion ∈ Mensaje_L
31:     Para i ← 0 hasta NUM_EVENT0 − 1 Hacer
32:         indice_elegido ← Aleatorio() MOD NUM_NODOS
33:         puerto_elegido ← 3000 + indice_elegido
34:         LimpiarMemoria(inyeccion)
35:         inyeccion.id_proceso ← 99
36:         inyeccion.reloj ← 0                                ▷ El proceso padre no altera el tiempo
37:         Si (Aleatorio() MOD 2) = 1 Entonces
38:             CopiarCadena(inyeccion.peticion, CMD_EN)
39:         Sino
40:             CopiarCadena(inyeccion.peticion, CMD_IN)
41:         Fin Si
42:         Escribir("\n[PADRE] Ordenando a la computadora ", (indice_elegido + 1) * 5, " con
el puerto ", puerto_elegido, " a realizar un evento...")
43:         enviar_msj(puerto_elegido, inyeccion)
44:         DormirMicrosegundos(100000)
45:     Fin Para

```

```
46:      Escribir("\n[PADRE] Simulación terminada. Esperando a que la red se estabilice...")
47:      Escribir("\n=====")
48:      Escribir(" [PADRE] INICIANDO PROTOCOLO DE APAGADO GLOBAL ")
49:      Escribir("=====\\n")
50:      Para  $i \leftarrow 0$  hasta  $NUM\_NODOS - 1$  Hacer
51:          LimpiarMemoria(inyeccion)
52:          inyeccion.id_proceso  $\leftarrow 99$ 
53:          inyeccion.reloj  $\leftarrow 0$ 
54:          CopiarCadena(inyeccion.peticion, CMD_EX)
55:          Escribir("\n[PADRE] Enviando comando de apagado a puerto ",  $3000 + i$ , "...")
56:          enviar_msj( $3000 + i$ , inyeccion)
57:          DormirMicrosegundos(50000)
58:      Fin Para
59:      EsperarHijo()
60:      Escribir("\n[SISTEMA GLOBAL] Todos los nodos P2P han finalizado exitosamente.
Fin del programa.")
61:  Fin Si
62:  Salir 0
63: Fin Procedimiento
```

Pseudocódigo: cnx_lamport.c (Conexión P2P)

Algoritmo Conexion_P2P

Constantes y Variables Globales

```
1: Definir  $PORT \in \mathbb{N}$ ,  $PORT \leftarrow 0$ 
2: Definir local, externo  $\in Mensaje\_L$ 
3: Definir puertos  $\in \mathbb{Z}[NUM\_NODOS]$ 
4: Definir encendido  $\leftarrow 1$ 
5: Definir enviar  $\leftarrow 0$ 

6: Procedimiento MAYOR( $a, b$ )
7:   Si  $a > b$  Entonces
8:     Salir  $a$ 
9:   Sino
10:    Salir  $b$ 
11:   Fin Si
12: Fin Procedimiento

13: Procedimiento CONEXION_P2P(user, dir)
14:   Definir thread_id  $\in Hilo$ 
15:   CopiarMemoria(puertos, dir, sizeof(puertos))
```

```

16:  PORT  $\leftarrow$  dir[user.indice]
17:  local.id_proceso  $\leftarrow$  user.id_proceso
18:  local.indice  $\leftarrow$  user.indice
19:  local.reloj  $\leftarrow$  local.reloj + 1 ▷ Inicialización del reloj local
20:  CrearHilo(thread_id, hilo_escucha, NULL)
21:  GenerarSemilla(Tiempo_Actual() XOR ObtenerPID())
22:  Mientras encendido Hacer
23:      Si enviar = 1 Entonces
24:          enviar  $\leftarrow$  0
25:          local.reloj  $\leftarrow$  local.reloj + 1 ▷ REGLA LAMPORT: Avanzar reloj antes de enviar
26:          Definir indice_new  $\in \mathbb{Z}$ 
27:          Repetir
28:              indice_new  $\leftarrow$  Aleatorio() MOD NUM_NODOS
29:          Hasta que indice_new  $\neq$  local.indice
30:          Escribir("\n[C", local.id_proceso, "] Enviare mensaje a la computadora con puerto
    ", puertos[indice_new], "...")
31:          LimpiarMemoria(local.peticion)
32:          FormatearCadena(local.peticion, "Soy la computadora %02d", local.id_proceso)
33:          enviar_msj(puertos[indice_new], local)
34:      Fin Si
35:  Fin Mientras
36:  EsperarHilo(thread_id)
37: Fin Procedimiento

38: Procedimiento HILO_ESCUCHA(arg)
39:  Definir server_fd, new_socket  $\in \mathbb{Z}$ 
40:  Definir address  $\in$  sockaddr_in
41:  Definir opt  $\leftarrow$  1
42:  server_fd  $\leftarrow$  socket(AF_INET, SOCK_STREAM, 0)
43:  setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, opt)
44:  address.sin_family  $\leftarrow$  AF_INET
45:  address.sin_addr.s_addr  $\leftarrow$  INADDR_ANY
46:  address.sin_port  $\leftarrow$  htons(PORT)
47:  bind(server_fd, address)
48:  listen(server_fd, 3)
49:  Escribir("[C", local.id_proceso, "] Escuchando Mensaje_L en puerto ", PORT, "...")
50:  Mientras encendido Hacer
51:      addrlen  $\leftarrow$  sizeof(address)
52:      new_socket  $\leftarrow$  accept(server_fd, address, addrlen)
53:      Si new_socket < 0 Entonces

```

```

54:      Romper
55:      Fin Si
56:      LimpiarMemoria(externo)
57:      Leer(new_socket, externo)
58:      Si externo.id_proceso = 99 Entonces                                ▷ Mensajes del Padre (Control)
59:          Si CompararCadena(externo.peticion, CMD_EX) = 0 Entonces
60:              Escribir("\n[C", local.id_proceso, "] Apagando por comando remoto...")
61:              encendido ← 0
62:          Sino Si CompararCadena(externo.peticion, CMD_EN) = 0 Entonces
63:              enviar ← 1
64:          Sino Si CompararCadena(externo.peticion, CMD_IN) = 0 Entonces
65:              Escribir("\n[C", local.id_proceso, "] -¿ EVENTO INTERNO ¿- Ejecutando tarea
local...")
66:              local.reloj ← local.reloj + 1
67:          Fin Si
68:      Sino                                ▷ Mensajes de otros Nodos P2P
69:          local.reloj ← mayor(local.reloj, externo.reloj) + 1    ▷ REGLA LAMPORT: Al
recibir
70:      Fin Si
71:      Escribir("\nInicio -¿ Mensaje Computadora [C", local.id_proceso, "] ¿-")
72:      Escribir("— ID_proceso: ", externo.id_proceso)
73:      Escribir("— Peticion: ", externo.peticion)
74:      Escribir("— Reloj Externo (Ts): ", externo.reloj)
75:      Escribir("— Reloj Local Actualizado: ", local.reloj)
76:      Escribir("Fin -¿ Mensaje Computadora [C", local.id_proceso, "] ¿-\n")
77:      Cerrar(new_socket)
78:      Fin Mientras
79:      Cerrar(server_fd)
80:      Salir NULL
81: Fin Procedimiento

82: Procedimiento ENVIAR_MSJ(puerto_destino, datos)
83:     Definir sock ∈ ℤ, sock ← 0
84:     Definir serv_addr ∈ sockaddr_in
85:     Si (sock ← socket(AF_INET, SOCK_STREAM, 0)) < 0 Entonces
86:         Salir
87:     Fin Si
88:     serv_addr.sin_family ← AF_INET
89:     serv_addr.sin_port ← htons(puerto_destino)
90:     inet_pton(AF_INET, DIRECTION, serv_addr.sin_addr)

```

```
91:  Si connect(sock, serv_addr) ; 0 Entonces
92:      Escribir("Error de conexion")
93:      Cerrar(sock)
94:      Salir
95:  Fin Si
96:  Enviar(sock, datos)
97:  Cerrar(sock)
98: Fin Procedimiento
```

3.1.7 Ejecución de Lamport.

```
1  [C05] Escuchando Mensaje_L en puerto 3000...
2  [C10] Escuchando Mensaje_L en puerto 3001...
3  [C15] Escuchando Mensaje_L en puerto 3002...
4
5  =====
6  [PADRE] INICIANDO SIMULACION P2P (RELOJES DE LAMPORT)
7  =====
8
9  [PADRE] Ordenando a la computadora 05 con el puerto 3000 a realizar un evento...
10
11 Inicio -> Mensaje Computadora [C05] <-
12 --- ID_proceso: 99
13 --- Peticion: ENVIAR
14 --- Reloj Externo (Ts): 0
15 --- Reloj Local Actualizado: 2
16 Fin -> Mensaje Computadora [C05] <-
17
18 [C05] Enviare mensaje a la computadora con puerto 3002...
19
20 Inicio -> Mensaje Computadora [C15] <-
21 --- ID_proceso: 5
22 --- Peticion: Soy la computadora 05
23 --- Reloj Externo (Ts): 2
24 --- Reloj Local Actualizado: 3
25 Fin -> Mensaje Computadora [C15] <-
26
27 [PADRE] Ordenando a la computadora 10 con el puerto 3001 a realizar un evento...
28
29 Inicio -> Mensaje Computadora [C10] <-
30 --- ID_proceso: 99
31 --- Peticion: ENVIAR
```

```
32 --- Reloj Externo (Ts): 0
33 --- Reloj Local Actualizado: 2
34 Fin -> Mensaje Computadora [C10] <-
35
36 [C10] Enviare mensaje a la computadora con puerto 3000...
37
38 Inicio -> Mensaje Computadora [C05] <-
39 --- ID_proceso: 10
40 --- Peticion: Soy la computadora 10
41 --- Reloj Externo (Ts): 2
42 --- Reloj Local Actualizado: 3
43 Fin -> Mensaje Computadora [C05] <-
44
45 [PADRE] Ordenando a la computadora 15 con el puerto 3002 a realizar un evento...
46
47 [C15] -> EVENTO INTERNO <- Ejecutando tarea local...
48
49 Inicio -> Mensaje Computadora [C15] <-
50 --- ID_proceso: 99
51 --- Peticion: LOCAL
52 --- Reloj Externo (Ts): 0
53 --- Reloj Local Actualizado: 4
54 Fin -> Mensaje Computadora [C15] <-
55
56 [PADRE] Ordenando a la computadora 05 con el puerto 3000 a realizar un evento...
57
58 Inicio -> Mensaje Computadora [C05] <-
59 --- ID_proceso: 99
60 --- Peticion: ENVIAR
61 --- Reloj Externo (Ts): 0
62 --- Reloj Local Actualizado: 4
63
64 [C05] Enviare mensaje a la computadora con puerto 3002...
65 Fin -> Mensaje Computadora [C05] <-
66
67 Inicio -> Mensaje Computadora [C15] <-
68 --- ID_proceso: 5
69 --- Peticion: Soy la computadora 05
70 --- Reloj Externo (Ts): 4
71 --- Reloj Local Actualizado: 5
72 Fin -> Mensaje Computadora [C15] <-
73
74 [PADRE] Ordenando a la computadora 05 con el puerto 3000 a realizar un evento...
```

```
75
76 [C05] -> EVENTO INTERNO <- Ejecutando tarea local...
77
78 Inicio -> Mensaje Computadora [C05] <-
79 --- ID_proceso: 99
80 --- Peticion: LOCAL
81 --- Reloj Externo (Ts): 0
82 --- Reloj Local Actualizado: 5
83 Fin -> Mensaje Computadora [C05] <-
84
85 [PADRE] Ordenando a la computadora 15 con el puerto 3002 a realizar un evento...
86
87 [C15] -> EVENTO INTERNO <- Ejecutando tarea local...
88
89 Inicio -> Mensaje Computadora [C15] <-
90 --- ID_proceso: 99
91 --- Peticion: LOCAL
92 --- Reloj Externo (Ts): 0
93 --- Reloj Local Actualizado: 6
94 Fin -> Mensaje Computadora [C15] <-
95
96 [PADRE] Ordenando a la computadora 10 con el puerto 3001 a realizar un evento...
97
98 Inicio -> Mensaje Computadora [C10] <-
99 --- ID_proceso: 99
100 --- Peticion: ENVIAR
101 --- Reloj Externo (Ts): 0
102 --- Reloj Local Actualizado: 3
103 Fin -> Mensaje Computadora [C10] <-
104
105 [C10] Enviare mensaje a la computadora con puerto 3000...
106
107 Inicio -> Mensaje Computadora [C05] <-
108 --- ID_proceso: 10
109 --- Peticion: Soy la computadora 10
110 --- Reloj Externo (Ts): 3
111 --- Reloj Local Actualizado: 6
112 Fin -> Mensaje Computadora [C05] <-
113
114 [PADRE] Simulación terminada. Esperando a que la red se estabilice...
115
116 =====
117 [PADRE] INICIANDO PROTOCOLO DE APAGADO GLOBAL
```

```
118 =====
119
120
121 [PADRE] Enviando comando de apagado a puerto 3000...
122
123 [C05] Apagando por comando remoto...
124
125 Inicio -> Mensaje Computadora [C05] <-
126 --- ID_proceso: 99
127 --- Peticion: EXIT
128 --- Reloj Externo (Ts): 0
129 --- Reloj Local Actualizado: 6
130 Fin -> Mensaje Computadora [C05] <-
131
132 [PADRE] Enviando comando de apagado a puerto 3001...
133
134 [C10] Apagando por comando remoto...
135
136 Inicio -> Mensaje Computadora [C10] <-
137 --- ID_proceso: 99
138 --- Peticion: EXIT
139 --- Reloj Externo (Ts): 0
140 --- Reloj Local Actualizado: 3
141 Fin -> Mensaje Computadora [C10] <-
142
143 [PADRE] Enviando comando de apagado a puerto 3002...
144
145 [C15] Apagando por comando remoto...
146
147 Inicio -> Mensaje Computadora [C15] <-
148 --- ID_proceso: 99
149 --- Peticion: EXIT
150 --- Reloj Externo (Ts): 0
151 --- Reloj Local Actualizado: 6
152 Fin -> Mensaje Computadora [C15] <-
153
154 [SISTEMA GLOBAL] Todos los nodos P2P han finalizado exitosamente. Fin del programa.
```

3.2 Problema 2: Algoritmo de Sincronización Vectorial

El objetivo de esta implementación es desarrollar el algoritmo de **Fidge-Mattern**, comúnmente conocido como relojes vectoriales. A diferencia del método escalar, este algoritmo permite capturar relaciones de causalidad de manera precisa, proporcionando a cada proceso una visión global del avance temporal de

todos los integrantes de la red distribuida.

3.2.1 Lógica de Actualización de Reloj Vectorial

En el archivo `cnx_vectores.c` se implementa la lógica de sincronización vectorial. La principal innovación de este enfoque radica en que cada proceso, en lugar de gestionar un único contador, mantiene un arreglo (vector) de tamaño n , donde cada posición i representa el conocimiento del proceso sobre el reloj del nodo j .

Al interactuar con otros nodos, el proceso no envía una estampa de tiempo aislada, sino su vector completo. Esto permite que el nodo receptor actualice su estado interno comparando cada componente de su vector con el recibido, tomando el valor máximo en cada posición:

$$V_i[j] = \max(V_i[j], V_{mensaje}[j]) \quad \forall j \in \{1, \dots, n\}$$

Finalmente, para representar el evento de recepción o cualquier evento local, el proceso incrementa únicamente su propia posición en el vector ($V_i[i]$), manteniendo así la trazabilidad causal de sus acciones.

- **Causalidad:** Permite identificar si un evento a ocurrió antes que b , o si son independientes.
- **Estructura:** El vector de enteros actúa como la memoria histórica de las interacciones del nodo.

3.2.2 Arquitectura del Sistema

La arquitectura descentralizada y multihilo diseñada para el algoritmo de Lamport se mantiene como base estructural para esta implementación. El sistema continúa operando mediante un proceso padre que orquesta la simulación y nodos P2P (con roles duales de cliente y servidor) que interactúan a través de sockets TCP.

La diferencia fundamental radica en la estructura de control para el seguimiento del tiempo lógico. Para soportar la causalidad, se realizaron adaptaciones clave en los componentes del sistema:

- **Estructura de Datos (`vector.h`):** Se reemplaza el reloj escalar por un arreglo de enteros (`int vector[NUM_NODOS]`). Esto permite que, en cada transmisión, el nodo emisor adjunte su “mapa causal” completo al mensaje.
- **Gestor P2P (`cnx_vectores.c`):** Las funciones de comunicación, especialmente el `hilo_escucha`, fueron modificadas. Ahora, en lugar de una simple comparación de enteros, el sistema itera sobre los vectores recibidos para aplicar la métrica de Fidge-Mattern índice por índice. Además, se añadió una función utilitaria (`imprimeV`) para visualizar el estado del vector de forma clara en la terminal.
- **Simulador de Red (`sinc_vectoriales.c`):** El proceso de instanciación (mediante `fork`) se adaptó para inicializar explícitamente todo el arreglo vectorial en ceros utilizando `memset`, garantizando un estado base limpio antes de comenzar la inyección de eventos aleatorios.

3.2.3 Estructura de Datos.

Para la comunicación, se corrige la estructura del mensaje `Mensaje_L` a `Mensaje_V` en el archivo de cabecera `vector.h`

```
1 typedef struct {
2     int id_proceso;           // ID del proceso que envia el mensaje
3     int indice;              // Posición de este proceso dentro del arreglo vectorial
4     int vector[NUM_NODOS];   // El mapa causal completo: [ C0, C1, C2 ]
5     char peticion[200];      // Texto del mensaje a enviar
6 } Mensaje_V;
```

3.2.4 Archivo Principal. (`sinc_vectoriales.c`)

Este archivo retoma la arquitectura de gestión de procesos empleada en la implementación de Lamport, adaptándola a la naturaleza multidimensional de los relojes vectoriales. La lógica principal reside en que, en lugar de un único contador, el sistema gestiona un vector de N componentes que representa el conocimiento temporal de cada nodo sobre el resto de la red. Esta estructura se transmite íntegramente en cada evento de envío, permitiendo que el receptor compare su estado local con el recibido para determinar relaciones de causalidad y concurrencia.

La ejecución se divide en tres niveles jerárquicos mediante el uso de `fork()`:

1. **Proceso Padre (Simulador de Caos):** Actúa como el plano de control externo. Su función es inyectar eventos aleatorios (internos o de envío) y coordinar el apagado global. Utiliza el ID 99 para que sus mensajes no alteren los relojes lógicos de los nodos, permitiendo una simulación limpia.
2. **Proceso Hijo (Coordinador):** Es el encargado de levantar la infraestructura de red, definiendo la agenda global de puertos (`directorio`) y supervisando el ciclo de vida de los nodos.
3. **Procesos Nietos (Nodos P2P):** Representan los nodos reales de la red. Cada uno inicializa su vector en cero mediante `memset` y comienza su ejecución invocando a `conexion_p2p`.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <sys/wait.h>
6 #include <time.h>
7 #include "vector.h"
8
9 Mensaje_V instancia;
10 int directorio[NUM_NODOS]; // Agenda global de la red
11
```

```
12 int main()
13 {
14     pid_t pid = fork();
15     if(pid < 0) exit(1);
16
17     // PROCESO HIJO (COORDINADOR): Levanta la infraestructura de red P2P
18     if(pid == 0)
19     {
20         for(int i = 0; i < NUM_NODOS; i++) directorio[i] = 3000 + i;
21         fflush(stdout);
22
23         for(int i = 0; i < NUM_NODOS; i++)
24         {
25             if(fork() == 0)
26             {
27                 // Configuración de fábrica del nodo
28                 instancia.id_proceso = 5 * (i + 1);
29                 instancia.indice = i;
30                 memset(instancia.vector, 0, sizeof(instancia.vector));
31                 memset(instancia.peticion, 0, sizeof(instancia.peticion));
32
33                 conexion_p2p(instancia, directorio);
34                 exit(0);
35             }
36         }
37         for(int i = 0; i < NUM_NODOS; i++){ wait(NULL); }
38         exit(0);
39     }
40
41     // PROCESO PADRE (SIMULADOR DE CAOS): Inyecta los eventos aleatoriamente
42     else
43     {
44         srand(time(NULL) ^ getpid() << 16);
45         sleep(1); // Damos 1 segundo para levantar sockets
46
47         printf("\n===== \n");
48         printf(" [PADRE] INICIANDO SIMULACION P2P (RELOJES VECTORIALES)\n");
49         printf("===== \n");
50
51         Mensaje_V inyeccion;
52
53         for(int i = 0; i < NUM_EVENTO; i++)
54         {
```

```
55     int indice_elegido = rand() % NUM_NODOS;
56     int puerto_elegido = 3000 + indice_elegido;
57
58     memset(&inyeccion, 0, sizeof(Mensaje_V));
59
60     // El Padre se identifica como "99".
61     // El memset previo asegura que su vector enviado es [0,0,0].
62     // Este proceso inyecta las instrucciones sin afectar los relojes
63     inyeccion.id_proceso = 99;
64     if (rand() % 2 == 1) strcpy(inyeccion.peticion, CMD_EN);
65     else strcpy(inyeccion.peticion, CMD_IN);
66
67     printf("\n[PADRE] Ordenando a la computadora C%02d (Puerto %d) que actúe...\n",
68           (indice_elegido + 1) * 5, puerto_elegido);
69
70     enviar_msj(puerto_elegido, inyeccion);
71     usleep(100000); // Ritmo al que se inyecta los eventos de envio o internos
72 }
73
74 printf("\n[PADRE] Simulación terminada. Esperando a que la red se estabilice...\n");
75
76 // APAGADO GLOBAL
77 printf("\n=====\n");
78 printf(" [PADRE] INICIANDO PROTOCOLO DE APAGADO GLOBAL \n");
79 printf("=====\n\n");
80
81 for(int i = 0; i < NUM_NODOS; i++)
82 {
83     memset(&inyeccion, 0, sizeof(Mensaje_V));
84     inyeccion.id_proceso = 99;
85     // Vector vacío garantizado por el memset
86     strcpy(inyeccion.peticion, CMD_EX);
87
88     printf("[PADRE] Enviando comando de apagado a puerto %d...\n", 3000 + i);
89     enviar_msj(3000 + i, inyeccion);
90     usleep(50000);
91 }
92
93 wait(NULL);
94 printf("\n[SISTEMA GLOBAL] Todos los nodos P2P han finalizado exitosamente.\n");
95 printf("----> Simulacion Terminada.\n");
```

```
96     }
97     return 0;
98 }
```

- Archivo de Cabecera (**vector.h**)

Este archivo es el pilar de la comunicación en la red, ya que define la estructura de datos que permite el rastreo de la causalidad. A diferencia del algoritmo de Lamport, donde se transmitía un único entero, aquí se define la estructura **Mensaje_V**, la cual transporta el vector completo de relojes lógicos (**vector[NUM_NODOS]**).

Además de la estructura, el archivo centraliza las constantes de la simulación, como el número de nodos y los comandos de control (**EXIT**, **ENVIAR**, **LOCAL**), asegurando que tanto el proceso padre como los nodos hijos utilicen los mismos protocolos de comunicación. Se incluye también la declaración de la función **imprimeV**, esencial para la depuración y visualización del estado interno de los vectores en la terminal.

```
1  #ifndef VECTOR_H
2  #define VECTOR_H
3
4  #define NUM_NODOS 3    // Numero de computadoras a crear
5  #define NUM_EVENTO 7  // Numero de eventos a realizarse en la red
6  #define CMD_EX "EXIT" // Comando de apagado global
7  #define CMD_EN "ENVIAR" // Comando de inyección (Evento de Envio)
8  #define CMD_IN "LOCAL" // Comando de inyeccion (Evento interno)
9  #define DIRECTION "127.0.0.1" // Direccion IP local
10
11 // ----- Estructura del mensaje de vectores (Relojes Vectoriales) -----
12 // A diferencia de Lamport (escalar), aqui llevamos el registro de la causalidad
13 // de TODOS los procesos de la red.
14 typedef struct {
15     int id_proceso;          // ID del proceso que envia el mensaje
16     int indice;             // Posicion de este proceso dentro del vector
17     int vector[ NUM_NODOS ]; // El mapa causal completo: [ C0, C1, C2 ]
18     char peticion[200];      // Texto del mensaje a enviar
19 } Mensaje_V;
20
21 // ----- Declaraciones de funciones -----
22 int mayor(int a, int b);
23 void conexion_p2p(Mensaje_V user, int dir[]);
24 void enviar_msj(int puerto_destino, Mensaje_V datos);
25 void imprimeV(int arr[]);
26
27 #endif
```

3.2.5 Archivo de Conexión P2P (`cnx_vectores.c`)

Este archivo constituye la implementación operativa del nodo bajo el esquema de relojes vectoriales. Al igual que en la práctica anterior, se aprovecha la característica de memoria aislada que proporciona `fork()` para que cada nodo gestione su propio vector lógico de manera independiente. El motor del sistema se divide en las siguientes funciones clave:

- **Variables Globales y Estado:** Se definen estructuras para el estado local (`local`) y la recepción (`externo`), además de banderas de control (`encendido`, `enviar`) que permiten la comunicación entre los hilos de ejecución.
- **enviar_msj:** Función encargada de la apertura de sockets TCP y la transmisión de la estructura `Mensaje_V`. Es crucial notar que ahora se transmite el vector completo, permitiendo que el receptor obtenga el conocimiento global del emisor.
- **hilo_escucha (Recepción Causal):** Representa el núcleo del algoritmo. Cuando el nodo recibe un mensaje de un vecino, ejecuta un ciclo `for` para actualizar cada posición de su vector local con el valor máximo encontrado entre su estado actual y el recibido:

$$V_{local}[i] = \max(V_{local}[i], V_{externo}[i]) \quad \forall i$$

Tras esta sincronización, el nodo incrementa su propia posición en el vector para registrar el evento de recepción. Si el mensaje proviene del padre (ID 99), se interpretan los comandos de control sin alterar la causalidad global, a excepción de los eventos locales.

- **conexion_p2p (Motor del Nodo):** Orquesta la inicialización y el comportamiento de envío. Al encenderse, el nodo incrementa su posición en el vector como su primer evento interno. En el ciclo de envío, aplica la **regla de Fidge-Mattern**: incrementa su reloj local antes de difundir su vector a un nodo aleatorio de la red.
- **imprimeV:** Función utilitaria que permite visualizar en la terminal el estado de los relojes en formato de arreglo (`[ C0 C1 C2 ]`), facilitando la verificación de la concurrencia y el orden de los eventos durante la simulación.

```
1 // Fragmento critico de la actualizacion vectorial en el hilo_escucha
2 else
3 {
4     // Aplicacion de la metrica de Fidge-Mattern
5     for (int i = 0; i < NUM_NODOS; i++) {
6         local.vector[i] = mayor(local.vector[i], externo.vector[i]);
7     }
8     // Incremento por evento de recepcion
9     local.vector[local.indice]++;
10 }
```

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <arpa/inet.h>
6 #include <pthread.h>
7 #include <time.h>
8 #include "vector.h"
9
10 // Variables globales aisladas para cada proceso P2P gracias a fork()
11 int PORT = 0; // Puerto asignado a este nodo
12 Mensaje_V local; // Estructura que mantiene el vector y estado interno
13 Mensaje_V externo; // Buffer para recibir mensajes de la red
14 int puertos[NUM_NODOS]; // Directorio de la red P2P (Agenda de vecinos)
15
16 int encendido = 1; // Bandera de ciclo de vida del hilo principal
17 int enviar = 0; // Bandera disparadora de eventos de envío
18
19 // Función auxiliar para calcular el máximo entre dos números
20 int mayor(int a, int b){return a > b ? a : b;}
21
22 // Esta funcion se encargara de enviar un mensaje a un puerto en especifico
23 void enviar_msj(int puerto_destino, Mensaje_V datos) {
24     int sock = 0;
25     struct sockaddr_in serv_addr;
26     // Creacion del socket tipo TCP para garantizar el envio y recepcion de mensaje
27     if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) return;
28
29     serv_addr.sin_port = htons(puerto_destino);
30     serv_addr.sin_family = AF_INET;
31     inet_pton(AF_INET, DIRECTION, &serv_addr.sin_addr);
32
33     // Tratamos de conectarnos si no puede nos salimos del programa
34     if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
35         perror("Error de conexion");
36         close(sock);
37         return;
38     }
39     // Enviamos el mensaje al conectarnos en el puerto definido
40     send(sock, &datos, sizeof(Mensaje_V), 0);
41     // Cerramos el socket ya que enviamos.
42     close(sock);
43 }
```

```
44
45 // Hilo Servidor: Se encarga exclusivamente de escuchar y reaccionar
46 void *hilo_escucha(void *arg)
47 {
48     int server_fd, new_socket;
49     struct sockaddr_in address;
50     int opt = 1;
51
52     server_fd = socket(AF_INET, SOCK_STREAM, 0);
53     setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
54
55     address.sin_family = AF_INET;
56     address.sin_addr.s_addr = INADDR_ANY;
57     address.sin_port = htons(PORT);
58
59     bind(server_fd, (struct sockaddr *)&address, sizeof(address));
60     listen(server_fd, 3);
61
62     printf("[C%02d] Escuchando por el puerto %d...\n", local.id_proceso, puertos[local.
indice]);
63
64     while (encendido)
65     {
66         int addrlen = sizeof(address);
67         new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t *)&
addrlen);
68
69         if (new_socket < 0) break;
70
71         memset(&externo, 0, sizeof(Mensaje_V));
72         read(new_socket, &externo, sizeof(Mensaje_V));
73
74         // 1. EVALUAR PRIMERO EL PLANO DE CONTROL (Ordenes del Padre)
75         if (externo.id_proceso == 99)
76         {
77             if (strcmp(externo.peticion, CMD_EX) == 0) {
78                 printf("\n[C%02d] Apagando por comando remoto...\n\n", local.id_proceso)
;
79                 encendido = 0;
80             }
81             else if (strcmp(externo.peticion, CMD_EN) == 0) {
82                 enviar = 1;
83             }
84         }
85     }
86 }
```

```
84         else if (strcmp(externo.peticion, CMD_IN) == 0) {
85             printf("\n[C%02d] -> EVENTO INTERNO <- Ejecutando tarea local...\n",
local.id_proceso);
86             local.vector[local.indice]++; // Incrementamos antes de imprimir el log
87         }
88     }
89     // 2. EVALUAR EL PLANO DE DATOS (Red P2P Real)
90     else
91     {
92         for (int i = 0; i < NUM_NODOS; i++) {
93             local.vector[i] = mayor(local.vector[i], externo.vector[i]);
94         }
95         local.vector[local.indice]++;
96     }
97
98     // 3. SE IMPRIME LO QUE SE RECIBE
99     printf("\nInicio -> Evento en Computadora [C%02d] <-\n", local.id_proceso);
100    printf("---- ID del evento: %d --> %s\n", externo.id_proceso, (externo.id_proceso
== 99) ? "PROCESO PADRE" : "NODO VECINO");
101    printf("---- Peticion recibida: %s\n", externo.peticion);
102    printf("---- Reloj Externo: "); imprimeV(externo.vector);
103    printf("---- Reloj Local:   "); imprimeV(local.vector);
104    printf("Fin -> Evento en Computadora [C%02d] <-\n", local.id_proceso);
105
106    close(new_socket);
107 }
108 close(server_fd);
109 return NULL;
110 }
111
112 // Hilo Principal: Motor del nodo P2P
113 void conexion_p2p(Mensaje_V user, int dir[])
114 {
115     pthread_t thread_id;
116
117     // Inyección de configuración local (El genoma del nodo)
118     memcpy(&puertos, dir, sizeof(puertos));
119     PORT = dir[user.indice];
120     local.id_proceso = user.id_proceso;
121     local.indice = user.indice;
122
123     // Todo vector inicia en [0,0,0]. El encendido del proceso cuenta como el Evento 1
124     local.vector[local.indice]++;
```

```
125
126 // Desplegamos el "Oído" del nodo en un hilo concurrente
127 pthread_create(&thread_id, NULL, hilo_escucha, NULL);
128
129 srand(time(NULL) ^ getpid());
130
131 while (encendido)
132 {
133     if (enviar == 1)
134     {
135         enviar = 0;
136
137         // ALGORITMO DE FIDGE-MATTERN - REGLA DE ENVIO
138         // Todo evento local interno (como decidir enviar algo) avanza el reloj
139         propio
140         local.vector[local.indice]++;
141
142         // Decisión libre: El nodo decide aleatoriamente a quién contactar
143         int indice_new;
144         do {
145             indice_new = rand() % NUM_NODOS;
146         } while (indice_new == local.indice); // Evita hacerse ecos a sí mismo
147
148         printf("\n[C%02d] Enviare mensaje a la computadora %02d con puerto %d...\n",
149             local.id_proceso, (indice_new + 1) * 5, puertos[indice_new]);
150
151         memset(local.peticion, 0, sizeof(local.peticion));
152         snprintf(local.peticion, sizeof(local.peticion), "Soy la computadora %02d",
153             local.id_proceso);
154
155         // Transmisión asíncrona del conocimiento causal
156         enviar_msj(puertos[indice_new], local);
157     }
158     usleep(50000); // Pequeña pausa para no saturar el CPU en el ciclo while
159 }
160
161 // Función auxiliar de formateo visual
162 void imprimeV(int arr[])
163 {
164     printf("[");
165     for (int i = 0; i < NUM_NODOS; i++) { printf(" %d ", arr[i]); }
```

```
166     printf("]\n");
167 }
```

3.2.6 Pseudocódigo.

Pseudocódigo: sinc_vectoriales.c (P2P)

Algoritmo Sincronizacion_Vectorial

Constantes y Estructuras (vector.h)

- 1: **Definir** $NUM_NODOS \leftarrow 3$
- 2: **Definir** $NUM_EVENTO \leftarrow 7$
- 3: **Definir** $CMD_EX \leftarrow "EXIT"$
- 4: **Definir** $CMD_EN \leftarrow "ENVIAR"$
- 5: **Definir** $CMD_IN \leftarrow "LOCAL"$
- 6: **Definir** $DIRECTION \leftarrow "127.0.0.1"$
- 7: **Estructura** *Mensaje_V*
- 8: $id_proceso \in \mathbb{Z}$
- 9: $indice \in \mathbb{Z}$
- 10: $vector \in \mathbb{Z}[NUM_NODOS]$ ▷ El mapa causal completo
- 11: $peticion \in \text{Alfabeto}[200]$
- 12: **FinEstructura**

Variables Globales

- 1: **Definir** $instancia \in Mensaje_V$
- 2: **Definir** $directorio \in \mathbb{Z}[NUM_NODOS]$
- 1: **Procedimiento** MAIN
- 2: $pid \leftarrow fork()$
- 3: **Si** $pid < 0$ **Entonces**
- 4: **Salir**(1)
- 5: **Fin Si**
- 6: **Si** $pid = 0$ **Entonces** ▷ Proceso hijo creador de nodos P2P
- 7: **Para** $i \leftarrow 0$ **hasta** $NUM_NODOS - 1$ **Hacer**
- 8: $directorio[i] \leftarrow 3000 + i$
- 9: **Fin Para**
- 10: **Para** $i \leftarrow 0$ **hasta** $NUM_NODOS - 1$ **Hacer**
- 11: **Si** $fork() = 0$ **Entonces** ▷ Proceso nieto (Nodo P2P individual)
- 12: $instancia.id_proceso \leftarrow 5 * (i + 1)$
- 13: $instancia.indice \leftarrow i$
- 14: **LimpiarMemoria**($instancia.vector$) ▷ Vector inicializado en 0

```

15:      LimpiarMemoria(instancia.peticion)
16:      conexion_p2p(instancia, directorio)
17:      Salir(0)
18:      Fin Si
19:      Fin Para
20:      Para  $i \leftarrow 0$  hasta  $NUM\_NODOS - 1$  Hacer
21:          EsperarHijo()
22:      Fin Para
23:      Salir(0)
24:      Sino                                     ▷ Proceso padre (Simulador de Caos)
25:          GenerarSemilla(TiempoActual() XOR (ObtenerPID()  $\ll$  16))
26:          DormirSegundos(1)
27:          Escribir("=====")
28:          Escribir(" [PADRE] INICIANDO SIMULACIÓN P2P (RELOJES VECTORIALES)
29:      ")
29:          Escribir("=====")
30:      Definir inyeccion  $\in$  Mensaje_V
31:      Para  $i \leftarrow 0$  hasta  $NUM\_EVENTO - 1$  Hacer
32:          indice_elegido  $\leftarrow$  Aleatorio() MOD  $NUM\_NODOS$ 
33:          puerto_elegido  $\leftarrow$   $3000 + indice\_elegido$ 
34:          LimpiarMemoria(inyeccion)                                     ▷ Garantiza vector [0,0,0]
35:          inyeccion.id_proceso  $\leftarrow$  99
36:          Si (Aleatorio() MOD 2) = 1 Entonces
37:              CopiarCadena(inyeccion.peticion, CMD_EN)
38:          Sino
39:              CopiarCadena(inyeccion.peticion, CMD_IN)
40:          Fin Si
41:          Escribir("\n[PADRE] Ordenando a la computadora ", (indice_elegido + 1) * 5, " que
42:      actue...")
42:          enviar_msj(puerto_elegido, inyeccion)
43:          DormirMicrosegundos(100000)
44:      Fin Para
45:      Escribir("\n[PADRE] Simulación terminada. Esperando a que la red se estabilice...")
46:      DormirSegundos(1)
47:      Escribir("\n=====")
48:      Escribir(" [PADRE] INICIANDO PROTOCOLO DE APAGADO GLOBAL ")
49:      Escribir("=====\n")
50:      Para  $i \leftarrow 0$  hasta  $NUM\_NODOS - 1$  Hacer
51:          LimpiarMemoria(inyeccion)

```

```
52:         inyeccion.id_proceso  $\leftarrow$  99
53:         CopiarCadena(inyeccion.peticion, CMD_EX)
54:         Escribir("\n[PADRE] Enviando comando de apagado a puerto ", 3000 + i, "...")
55:         enviar_msj(3000 + i, inyeccion)
56:         DormirMicrosegundos(50000)
57:     Fin Para
58:     EsperarHijo()
59:     Escribir("\n[SISTEMA GLOBAL] Todos los nodos P2P han finalizado exitosamente.")
60: Fin Si
61: Salir 0
62: Fin Procedimiento
```

Pseudocódigo: cnx_vectores.c (Conexión P2P Vectorial)

Algoritmo Conexion_P2P_Vectorial

Constantes y Variables Globales

```
1: Definir PORT  $\in \mathbb{N}$ , PORT  $\leftarrow$  0
2: Definir local, externo  $\in$  Mensaje_V
3: Definir puertos  $\in \mathbb{Z}[\text{NUM\_NODOS}]$ 
4: Definir encendido  $\leftarrow$  1
5: Definir enviar  $\leftarrow$  0

6: Procedimiento MAYOR(a, b)
7:     Si a > b Entonces
8:         Salir a
9:     Sino
10:        Salir b
11:    Fin Si
12: Fin Procedimiento

13: Procedimiento CONEXION_P2P(user, dir)
14:    Definir thread_id  $\in$  Hilo
15:    CopiarMemoria(puertos, dir, sizeof(puertos))
16:    PORT  $\leftarrow$  dir[user.indice]
17:    local.id_proceso  $\leftarrow$  user.id_proceso
18:    local.indice  $\leftarrow$  user.indice
19:    local.vector[local.indice]  $\leftarrow$  local.vector[local.indice] + 1     $\triangleright$  Evento 1: Encendido del
    proceso
20:    CrearHilo(thread_id, hilo_escucha, NULL)
21:    GenerarSemilla(Tiempo_Actual() XOR ObtenerPID())
22:    Mientras encendido Hacer
```

```

23:      Si enviar = 1 Entonces
24:          enviar  $\leftarrow$  0
25:          local.vector[local.indice]  $\leftarrow$  local.vector[local.indice] + 1          ▷ REGLA
FIDGE-MATTERN: Evento interno
26:      Definir indice_new  $\in \mathbb{Z}$ 
27:      Repetir
28:          indice_new  $\leftarrow$  Aleatorio() MOD NUM_NODOS
29:      Hasta que indice_new  $\neq$  local.indice          ▷ Evita hacerse eco a sí mismo
30:      Escribir("\n[C", local.id_proceso, "] Enviare mensaje a la computadora con puerto
", puertos[indice_new], "...")
31:      LimpiarMemoria(local.peticion)
32:      FormatearCadena(local.peticion, "Soy la computadora %02d", local.id_proceso)
33:      enviar_msj(puertos[indice_new], local)
34:      Fin Si
35:      DormirMicrosegundos(50000)
36:      Fin Mientras
37:      EsperarHilo(thread_id)
38:  Fin Procedimiento

39:  Procedimiento HILO_ESCUCHA(arg)
40:      Definir server_fd, new_socket  $\in \mathbb{Z}$ 
41:      Definir address  $\in$  sockaddr_in
42:      Definir opt  $\leftarrow$  1
43:      server_fd  $\leftarrow$  socket(AF_INET, SOCK_STREAM, 0)
44:      setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, opt)
45:      address.sin_family  $\leftarrow$  AF_INET
46:      address.sin_addr.s_addr  $\leftarrow$  INADDR_ANY
47:      address.sin_port  $\leftarrow$  htons(PORT)
48:      bind(server_fd, address)
49:      listen(server_fd, 3)
50:      Mientras encendido Hacer
51:          addrlen  $\leftarrow$  sizeof(address)
52:          new_socket  $\leftarrow$  accept(server_fd, address, addrlen)
53:          Si new_socket < 0 Entonces
54:              Romper
55:          Fin Si
56:          LimpiarMemoria(externo)
57:          Leer(new_socket, externo)
58:          Si externo.id_proceso = 99 Entonces          ▷ Mensajes del Padre (Control)
59:              Si CompararCadena(externo.peticion, CMD_EX) = 0 Entonces

```

```

60:      encendido ← 0
61:      Sino Si CompararCadena(externo.peticion, CMD_EN) = 0 Entonces
62:      enviar ← 1
63:      Sino Si CompararCadena(externo.peticion, CMD_IN) = 0 Entonces
64:      local.vector[local.indice] ← local.vector[local.indice] + 1
65:      Fin Si
66:      Sino                                     ▷ Plano de Datos (Red P2P Real)
67:      Para i ← 0 hasta NUM_NODOS − 1 Hacer
68:      local.vector[i] ← mayor(local.vector[i], externo.vector[i])           ▷ REGLA
FIDGE-MATTERN: Actualizar componentes
69:      Fin Para
70:      local.vector[local.indice] ← local.vector[local.indice] + 1           ▷ Incremento por
recepción
71:      Fin Si
72:      Escribir("\nInicio -¿ Evento en Computadora [C", local.id_proceso, "] i-")
73:      Escribir("— ID del evento: ", externo.id_proceso)
74:      Escribir("— Peticion: ", externo.peticion)
75:      imprimeV(externo.vector)
76:      imprimeV(local.vector)
77:      Escribir("Fin -¿ Evento en Computadora [C", local.id_proceso, "] i-\n")
78:      Cerrar(new_socket)
79:      Fin Mientras
80:      Cerrar(server_fd)
81:      Salir NULL
82: Fin Procedimiento

83: Procedimiento IMPRIMEV(arr)
84:   EscribirSinSalto("[")
85:   Para i ← 0 hasta NUM_NODOS − 1 Hacer
86:     EscribirSinSalto(" ", arr[i], " ")
87:   Fin Para
88:   Escribir("]")
89: Fin Procedimiento

```

3.2.7 Ejecución de Vectoriales.

```

1 [C05] Escuchando por el puerto 3000...
2 [C10] Escuchando por el puerto 3001...
3 [C15] Escuchando por el puerto 3002...
4
5 =====

```

```
6  [PADRE] INICIANDO SIMULACION P2P (RELOJES VECTORIALES)
7  =====
8
9  [PADRE] Ordenando a la computadora C05 (Puerto 3000) que actúe...
10
11 Inicio -> Evento en Computadora [C05] <-
12 --- ID del evento: 99 --> PROCESO PADRE
13 --- Peticion recibida: ENVIAR
14 --- Reloj Externo: [ 0  0  0 ]
15 --- Reloj Local:   [ 1  0  0 ]
16 Fin -> Evento en Computadora [C05] <-
17
18 [C05] Enviare mensaje a la computadora 10 con puerto 3001...
19
20 Inicio -> Evento en Computadora [C10] <-
21 --- ID del evento: 5 --> NODO VECINO
22 --- Peticion recibida: Soy la computadora 05
23 --- Reloj Externo: [ 2  0  0 ]
24 --- Reloj Local:   [ 2  2  0 ]
25 Fin -> Evento en Computadora [C10] <-
26
27 [PADRE] Ordenando a la computadora C05 (Puerto 3000) que actúe...
28
29 [C05] -> EVENTO INTERNO <- Ejecutando tarea local...
30
31 Inicio -> Evento en Computadora [C05] <-
32 --- ID del evento: 99 --> PROCESO PADRE
33 --- Peticion recibida: LOCAL
34 --- Reloj Externo: [ 0  0  0 ]
35 --- Reloj Local:   [ 3  0  0 ]
36 Fin -> Evento en Computadora [C05] <-
37
38 [PADRE] Ordenando a la computadora C15 (Puerto 3002) que actúe...
39
40 Inicio -> Evento en Computadora [C15] <-
41 --- ID del evento: 99 --> PROCESO PADRE
42 --- Peticion recibida: ENVIAR
43 --- Reloj Externo: [ 0  0  0 ]
44 --- Reloj Local:   [ 0  0  1 ]
45 Fin -> Evento en Computadora [C15] <-
46
47 [C15] Enviare mensaje a la computadora 05 con puerto 3000...
48
```

```
49 Inicio -> Evento en Computadora [C05] <-
50 --- ID del evento: 15 --> NODO VECINO
51 --- Peticion recibida: Soy la computadora 15
52 --- Reloj Externo: [ 0 0 2 ]
53 --- Reloj Local:   [ 4 0 2 ]
54 Fin -> Evento en Computadora [C05] <-
55
56 [PADRE] Ordenando a la computadora C15 (Puerto 3002) que actúe...
57
58 Inicio -> Evento en Computadora [C15] <-
59 --- ID del evento: 99 --> PROCESO PADRE
60 --- Peticion recibida: ENVIAR
61 --- Reloj Externo: [ 0 0 0 ]
62 --- Reloj Local:   [ 0 0 2 ]
63 Fin -> Evento en Computadora [C15] <-
64
65 [C15] Enviare mensaje a la computadora 10 con puerto 3001...
66
67 Inicio -> Evento en Computadora [C10] <-
68 --- ID del evento: 15 --> NODO VECINO
69 --- Peticion recibida: Soy la computadora 15
70 --- Reloj Externo: [ 0 0 3 ]
71 --- Reloj Local:   [ 2 3 3 ]
72 Fin -> Evento en Computadora [C10] <-
73
74 [PADRE] Ordenando a la computadora C15 (Puerto 3002) que actúe...
75
76 Inicio -> Evento en Computadora [C15] <-
77 --- ID del evento: 99 --> PROCESO PADRE
78 --- Peticion recibida: ENVIAR
79 --- Reloj Externo: [ 0 0 0 ]
80 --- Reloj Local:   [ 0 0 3 ]
81 Fin -> Evento en Computadora [C15] <-
82
83 [C15] Enviare mensaje a la computadora 10 con puerto 3001...
84
85 Inicio -> Evento en Computadora [C10] <-
86 --- ID del evento: 15 --> NODO VECINO
87 --- Peticion recibida: Soy la computadora 15
88 --- Reloj Externo: [ 0 0 4 ]
89 --- Reloj Local:   [ 2 4 4 ]
90 Fin -> Evento en Computadora [C10] <-
91
```

```
92 [PADRE] Ordenando a la computadora C15 (Puerto 3002) que actúe...
93
94 Inicio -> Evento en Computadora [C15] <-
95 --- ID del evento: 99 --> PROCESO PADRE
96 --- Peticion recibida: ENVIAR
97 --- Reloj Externo: [ 0 0 0 ]
98 --- Reloj Local:   [ 0 0 4 ]
99 Fin -> Evento en Computadora [C15] <-
100
101 [C15] Enviare mensaje a la computadora 10 con puerto 3001...
102
103 Inicio -> Evento en Computadora [C10] <-
104 --- ID del evento: 15 --> NODO VECINO
105 --- Peticion recibida: Soy la computadora 15
106 --- Reloj Externo: [ 0 0 5 ]
107 --- Reloj Local:   [ 2 5 5 ]
108 Fin -> Evento en Computadora [C10] <-
109
110 [PADRE] Ordenando a la computadora C05 (Puerto 3000) que actúe...
111
112 Inicio -> Evento en Computadora [C05] <-
113 --- ID del evento: 99 --> PROCESO PADRE
114 --- Peticion recibida: ENVIAR
115 --- Reloj Externo: [ 0 0 0 ]
116 --- Reloj Local:   [ 4 0 2 ]
117 Fin -> Evento en Computadora [C05] <-
118
119 [C05] Enviare mensaje a la computadora 15 con puerto 3002...
120
121 Inicio -> Evento en Computadora [C15] <-
122 --- ID del evento: 5 --> NODO VECINO
123 --- Peticion recibida: Soy la computadora 05
124 --- Reloj Externo: [ 5 0 2 ]
125 --- Reloj Local:   [ 5 0 6 ]
126 Fin -> Evento en Computadora [C15] <-
127
128 [PADRE] Simulación terminada. Esperando a que la red se estabilice...
129
130 =====
131 [PADRE] INICIANDO PROTOCOLO DE APAGADO GLOBAL
132 =====
133
134 [PADRE] Enviando comando de apagado a puerto 3000...
```

```
135
136 [C05] Apagando por comando remoto...
137
138
139 Inicio -> Evento en Computadora [C05] <-
140 --- ID del evento: 99 --> PROCESO PADRE
141 --- Peticion recibida: EXIT
142 --- Reloj Externo: [ 0 0 0 ]
143 --- Reloj Local:   [ 5 0 2 ]
144 Fin -> Evento en Computadora [C05] <-
145 [PADRE] Enviando comando de apagado a puerto 3001...
146
147 [C10] Apagando por comando remoto...
148
149
150 Inicio -> Evento en Computadora [C10] <-
151 --- ID del evento: 99 --> PROCESO PADRE
152 --- Peticion recibida: EXIT
153 --- Reloj Externo: [ 0 0 0 ]
154 --- Reloj Local:   [ 2 5 5 ]
155 Fin -> Evento en Computadora [C10] <-
156 [PADRE] Enviando comando de apagado a puerto 3002...
157
158 [C15] Apagando por comando remoto...
159
160
161 Inicio -> Evento en Computadora [C15] <-
162 --- ID del evento: 99 --> PROCESO PADRE
163 --- Peticion recibida: EXIT
164 --- Reloj Externo: [ 0 0 0 ]
165 --- Reloj Local:   [ 5 0 6 ]
166 Fin -> Evento en Computadora [C15] <-
167
168 [SISTEMA GLOBAL] Todos los nodos P2P han finalizado exitosamente.
169 ----> Simulacion Terminada.
```

3.3 Algoritmo de Sincronización de Cristian

3.3.1 Lógica de Actualización de Relojes Físicos

El algoritmo de Cristian es un método probabilístico diseñado para sincronizar relojes físicos en sistemas distribuidos. Su premisa se basa en calcular el tiempo de tránsito de un mensaje por la red, asumiendo que el viaje de ida tarda aproximadamente lo mismo que el de vuelta (simetría en la red).

Para lograr esta sincronización, el cliente registra el tiempo exacto en que envía una solicitud (T_0) y el tiempo en que recibe la respuesta (T_1). La diferencia entre ambos nos da el Tiempo de Ida y Vuelta o RTT (Round Trip Time):

$$RTT = T_1 - T_0$$

El servidor, al procesar la petición, inserta su propia estampa de tiempo oficial (T_s). Finalmente, el cliente ajusta su reloj local sumando al tiempo del servidor la mitad del RTT (que representa la latencia estimada de bajada):

$$\text{Hora Corregida} = T_s + \frac{RTT}{2}$$

Mecanismo de Precisión: En el archivo `sinc_cristian.c`, se implementó un umbral de tolerancia (`RTT_MAX_MS = 600`). Si el retardo supera este límite, significa que la red sufrió una congestión anómala, invalidando la asunción de simetría. En ese caso, el cliente descarta la estampa de tiempo y reintenta la operación.

3.3.2 Arquitectura del Sistema

La arquitectura abandona el esquema Peer-to-Peer (P2P) de los problemas anteriores para adoptar una estricta **topología en estrella** (Cliente-Servidor).

- **Protocolo de Transporte:** La comunicación se realiza mediante datagramas **UDP**(`SOCK_DGRAM`). Dado que se transmiten paquetes muy pequeños y sensibles a la latencia, UDP evita el costo temporal que implicaría establecer una conexión TCP.
- **Jerarquía de Procesos:** El proceso Padre asume el rol ininterrumpido de Servidor Maestro. El proceso Hijo actúa como coordinador, lanzando a los procesos nietos (quienes fungirán como clientes independientes) y, al finalizar todos, inyecta el comando **EXIT** para apagar el servidor.

3.3.3 Estructura de Datos (`header.h`)

Para manipular el tiempo físico sin sufrir desbordamientos aritméticos, se abstrajo la hora en estructuras específicas:

- **tiempo:** Fragmenta la hora local en horas, minutos, segundos y milisegundos, permitiendo una fácil lectura e impresión.
- **Mensaje_C:** Es la carga útil del datagrama UDP. A diferencia de las prácticas de relojes lógicos, aquí se transportan tres estampas de tiempo vitales (T_0 , T_s , T_1) que viajarán entre el cliente y el servidor.

```
1 // ----- Estructura para almacenar el tiempo físico y visual -----
2 typedef struct {
3     int horas;
4     int minutos;
5     int segundos;
```

```
6     int milisegundos;
7 } tiempo;
8
9 // ----- Estructura del mensaje de Cristian -----
10 typedef struct {
11     int id_proceso;
12     char peticion[200];
13     tiempo tiempo_envio;    // T0: Estampa de tiempo al enviar
14     tiempo tiempo_servidor; // Ts: Estampa oficial devuelta
15     tiempo tiempo_respuesta; // T1: Estampa al recibir respuesta
16 } Mensaje_C;
```

3.3.4 Archivo Principal (sinc_cristian.c)

El núcleo matemático del archivo consiste en la función `capturar_tiempo`, que extrae la hora real del sistema utilizando `CLOCK_REALTIME`, y la función `conversion_ms`, que transforma la estructura tridimensional de tiempo a un entero de 64 bits (`long long`). Esta conversión es obligatoria para poder realizar las restas y divisiones de la fórmula de Cristian a nivel de milisegundos sin pérdida de precisión.

```
1 #include <stdio.h>           // printf, perror
2 #include <stdlib.h>          // exit, rand, srand
3 #include <string.h>          // memset, strcpy, strcmp
4 #include <unistd.h>           // fork, sleep, usleep, getpid, close
5 #include <sys/wait.h>         // wait
6 #include <arpa/inet.h>        // inet_aton, htons
7 #include <netinet/in.h>       // sockaddr_in
8 #include <time.h>             // clock_gettime, localtime
9 #include "header.h"          // estructuras tiempo / Mensaje_C y constantes
10
11 // Variables globales para definir el limite en la respuesta
12 // Y para el retardo maximo del servidor a la hora de enviar respuestas.
13 #define RTT_MAX_MS  600      // 500 ms como límite razonable en loopback
14 #define DELAY_SERV  300000    // Retardo maximo del servidor en us
15 #define DELAY_CLI   250000    // Retardo maximo del cliente en us
16
17 // Esta funcion imprime una estructura tiempo con formato H:M:S:ms
18 void mostrar_hora(tiempo *t)
19 {
20     printf("[%d:%d:%d:%d]\n",    // formato H:M:S:ms
21         t->horas,
22         t->minutos,
23         t->segundos,
24         t->milisegundos);
25 }
```

```
26
27 // Convertir estructura tiempo a milisegundos como un entero de 64 bits
28 long long conversion_ms(tiempo *t)
29 {
30     // Usamos long long para numeros de 64 bits y poder tener mejor aproximacion
31     // Realizamos la conversion de horas, minutos y segundos a milisegundos
32     // 1 hora = 3600 segundos * 1000 ms = 3,600,000 ms
33     // 1 minuto = 60 segundos * 1000 ms = 60,000 ms
34     // 1 segundo = 1,000 ms
35     long long t_ms = (long long)t->horas * 3600000LL +
36                     (long long)t->minutos * 60000LL +
37                     (long long)t->segundos * 1000LL +
38                     t->milisegundos;
39     return t_ms;
40 }
41
42 // Lee el reloj del sistema en tiempo real y llena la estructura tiempo
43 void capturar_tiempo(tiempo *t)
44 {
45     struct timespec ts;           // estructura con segundos y nanosegundos
46     struct tm *tm_info;           // estructura para desglosar hora local
47
48     clock_gettime(CLOCK_REALTIME, &ts); // obtiene hora del reloj del sistema
49     tm_info = localtime(&ts.tv_sec);    // convierte segundos Unix a hora local
50
51     t->milisegundos = ts.tv_nsec / 1000000; // 1 000 000 ns = 1 ms
52     t->segundos     = tm_info->tm_sec;      // segundos (0-59)
53     t->minutos      = tm_info->tm_min;      // minutos (0-59)
54     t->horas        = tm_info->tm_hour;     // horas (0-23)
55 }
56
57 // El servidor se encarga de recibir el mensaje del cliente lo procesa y le regresa
58 // su respuesta con su estampa de tiempo al momento de recibir la solicitud
59 void ejecutar_servidor_C()
60 {
61     int desc_socket;               // descriptor del socket UDP del servidor
62     struct sockaddr_in direct;     // dirección propia del servidor
63     struct sockaddr_in cliente;    // dirección del cliente que llama
64     socklen_t tam_cliente;         // tamaño de la dirección del cliente
65     Mensaje_C msj;                // buffer para el mensaje recibido/enviado
66
67     printf("\n[SERVIDOR] Reloj Maestro iniciado en el puerto %d...\n", PUERTO);
68 }
```

```
69 desc_socket = socket(AF_INET, SOCK_DGRAM, 0); // socket UDP (sin conexión)
70
71 memset(&direct, 0, sizeof(direct)); // limpia la estructura de dirección
72 direct.sin_family = AF_INET; // familia IPv4
73 direct.sin_port = htons(PUERTO); // puerto en orden de red
74 direct.sin_addr.s_addr = INADDR_ANY; // acepta peticiones de cualquier IP
75
76 // Se asocia el socket a la dirección "escucha por el puerto"
77 bind(desc_socket, (struct sockaddr *)&direct, sizeof(direct));
78
79 while (1) // bucle infinito hasta recibir EXIT
80 {
81     tam_cliente = sizeof(cliente); // se debe resetear el tamaño antes de cada
recvfrom
82     memset(&msj, 0, sizeof(Mensaje_C)); // limpia el buffer del mensaje
83
84     // Se queda esperando hasta recibir un datagrama de algún cliente.
85     recvfrom(desc_socket, &msj, sizeof(Mensaje_C), 0, (struct sockaddr *)&cliente, &
tam_cliente);
86
87     if (strcmp(msj.peticion, "EXIT") == 0) // comando de apagado remoto
88     {
89         printf("\n[SERVIDOR] Comando EXIT recibido. Apagando.\n");
90         break; // sale del bucle y cierra el servidor
91     }
92     else if (strcmp(msj.peticion, "ACTUALIZAR") == 0) // petición de sincronización
93     {
94         // El tiempo del servidor se captura al momento de recibir la petición
95         capturar_tiempo(&msj.tiempo_servidor); //<- Ts capturado primero
96
97         // Simulamos latencia de procesamiento
98         usleep(rand() % DELAY_SERV);
99         // Mostramos sobre la terminal del servidor la hora en la que recibió la
petición con PID
100         printf("\n[SERVIDOR] Petición recibida de PID %d. Hora despachada: ", msj.
id_proceso);
101         // Se muestra la hora [H:M:S:ms]
102         mostrar_hora(&msj.tiempo_servidor);
103
104         strcpy(msj.peticion, "OK"); // cambia la petición a confirmación
105
```

```
106         // Devuelve el mensaje (con Ts dentro) al cliente que lo solicitó
107         sendto(desc_socket, &msj, sizeof(Mensaje_C), 0, (struct sockaddr *)&cliente,
tam_cliente);
108     }
109     // Regresamos a escuchar y limpiar la estructura.
110 }
111 // Si el comando EXIT se recibio, se debe cerrar el socket
112 close(desc_socket); // libera el descriptor del socket
113 }
114
115 // El cliente se encargara de solicitar al servidor para sincronizar su reloj
116 void ejecutar_cliente_C(int indice)
117 {
118     srand(time(NULL) ^ (getpid() << 16)); // semilla única por proceso
119     usleep(rand() % DELAY_CLI); // retardo aleatorio para escalonar los clientes
120
121     int desc_socket; // descriptor del socket UDP del cliente
122     struct sockaddr_in dest; // dirección del servidor destino
123     Mensaje_C msj; // buffer del mensaje enviado/recibido
124     tiempo reloj_sincronizado; // resultado final: hora ajustada por Cristian
125
126     desc_socket = socket(AF_INET, SOCK_DGRAM, 0); // socket UDP
127
128     memset(&dest, 0, sizeof(dest)); // limpia la estructura de dirección
129     dest.sin_family = AF_INET; // familia IPv4
130     dest.sin_port = htons(PUERTO); // puerto del servidor en orden de red
131     inet_aton(DIR_SERV, &dest.sin_addr); // convierte la IP del servidor a binario
132
133     // El algoritmo de Cristian especifica que si el RTT es demasiado grande,
134     // la estimación Ts + RTT/2 tiene demasiado error y el resultado no es
135     // confiable. En ese caso se debe repetir la petición hasta obtener
136     // un RTT dentro del umbral aceptable (RTT_MAX_MS).
137     long long rango; // RTT medido en este intento
138     int intentos = 0; // contador de intentos realizados
139
140     // Utilizamos un ciclo do-while por si la diferencia de tiempo es mayor al limite
    establecido
141     do
142     {
143         intentos++; // incrementa el contador de intentos
144
145         memset(&msj, 0, sizeof(Mensaje_C)); // limpia el mensaje antes de cada intento
146         msj.id_proceso = getpid(); // identifica al proceso cliente
```

```
147     strcpy(msj.peticion, "ACTUALIZAR");    // petición de sincronización al servidor
148
149     // PASO 1: Capturar T0 y enviar la petición
150     capturar_tiempo(&msj.tiempo_envio);    // T0: estampa de tiempo de salida
151     sendto(desc_socket, &msj, sizeof(Mensaje_C), 0, (struct sockaddr *)&dest, sizeof
(dest));
152
153     // PASO 2: Recibir la respuesta del servidor (Ts viaja dentro de msj)
154     socklen_t tam_struct = sizeof(dest);
155     recvfrom(desc_socket, &msj, sizeof(Mensaje_C), 0, (struct sockaddr *)&dest, &
tam_struct);
156
157     // PASO 3: Capturamos el tiempo en que llego la respuesta
158     capturar_tiempo(&msj.tiempo_respuesta); // T1: estampa de tiempo de llegada
159
160     // Convertimos el tiempo de envio T0 a milisegundos
161     long long t0_ms = conversion_ms(&msj.tiempo_envio);
162
163     // Convertimos el tiempo de respuesta T1 a milisegundos
164     long long t1_ms = conversion_ms(&msj.tiempo_respuesta);
165
166     rango = t1_ms - t0_ms; // RTT = T1 - T0 (diferencia de tiempo de ida y vuelta)
167
168     if (rango > RTT_MAX_MS) // si el rango supera el umbral muestra mensaje y
repite
169         printf("[CLIENTE #%02d] RTT = %lld ms demasiado alto, reintentando... (
intento %d)\n", indice + 1, rango, intentos);
170
171     } while (rango > RTT_MAX_MS); // repite hasta obtener un rango aceptable
172
173     // FORMULA DE CRISTIAN -> si la respuesta entre el servidor y el cliente estan
dentro del limite se corrige el reloj
174     // El reloj del servidor mas la diferencia de tiempo ida y vuelta del mensaje entre
2 (asumiendo simetria)
175     // hora_corregida = Ts + (T1 - T0) / 2
176     // La mitad del RTT aproxima el tiempo que tardó el mensaje en llegar desde el
servidor hasta el cliente
177
178     // Convertimos el tiempo del servidor Ts a milisegundos
179     long long ts_ms = conversion_ms(&msj.tiempo_servidor);
180
181     long long latencia_ms = rango / 2; // latencia estimada = RTT / 2
182     long long hora_corregida_ms = ts_ms + latencia_ms; // hora ajustada por Cristian
```

```
183
184 // Reconstruimos la hora corregida en la estructura tiempo (h:m:s:ms)
185 reloj_sincronizado.milisegundos = hora_corregida_ms % 1000; // residuo en
ms
186
187 long long seg_totales = hora_corregida_ms / 1000; // total en segundos
188 reloj_sincronizado.segundos = seg_totales % 60; // segundos (0-59)
189
190 long long min_totales = seg_totales / 60; // total en minutos
191 reloj_sincronizado.minutos = min_totales % 60; // minutos (0-59)
192
193 long long horas_totales = min_totales / 60; // total en horas
194 reloj_sincronizado.horas = horas_totales % 24; // horas formato 24h
195
196 // Imprimimos todos los tiempos y el resultado final
197 printf("\n[CLIENTE #%02d | PID %d] ---> T0 (Salida) : ", indice + 1, getpid());
198 mostrar_hora(&msj.tiempo_envio);
199
200 printf("[CLIENTE #%02d | PID %d] ---> T1 (Llegada) : ", indice + 1, getpid());
201 mostrar_hora(&msj.tiempo_respuesta);
202
203 printf("[CLIENTE #%02d | PID %d] ---> Ts (Servidor) : ", indice + 1, getpid());
204 mostrar_hora(&msj.tiempo_servidor);
205
206 printf("[CLIENTE #%02d | PID %d] ---> RTT : %lld ms (intento(s): %d)\n",
indice + 1, getpid(), rango, intentos); // muestra el RTT final aceptado
207
208 printf("[CLIENTE #%02d | PID %d] ---> Latencia Calc : %lld ms\n", indice + 1,
getpid(), latencia_ms);
209
210 printf("[CLIENTE #%02d | PID %d] ---> RELOJ AJUSTADO : ", indice + 1, getpid());
211 mostrar_hora(&reloj_sincronizado);
212
213 close(desc_socket); // libera el descriptor del socket
214 usleep(1000000); // espera 1 s para que el servidor procese antes de salir
215 exit(0); // el proceso nieta termina limpiamente
216 }
217
218 int main()
219 {
220 pid_t pid = fork(); // se crea el proceso hijo (los nietos seran clientes)
221
222 if (pid < 0) exit(1); // error al hacer fork
```

```
223
224 // Proceso padre actuara como el servidor
225 if (pid > 0)
226 {
227     ejecutar_servidor_C(); // Entra aquí y sale hasta recibir EXIT
228
229     int estado;
230     waitpid(pid, &estado, 0); // espera específicamente al hijo coordinador
231
232     printf("\n[SISTEMA] Simulación finalizada.\n");
233     exit(0);
234 }
235
236 // PROCESO HIJO: coordina y lanza los clientes
237 else
238 {
239     sleep(1); // espera a que el servidor esté listo
240
241     for (int i = 0; i < N_CLIENTES; i++) // lanza N_CLIENTES procesos nietos
242     {
243         if (fork() == 0) // cada nieto es un cliente
244             ejecutar_cliente_C(i); // ejecuta el algoritmo de Cristian
245         y hace exit(0)
246     }
247     // El proceso hijo espera a que todos los nietos terminen (clientes).
248     for (int i = 0; i < N_CLIENTES; i++) wait(NULL);
249
250     // Todos los clientes terminaron: ordenar el apagado del servidor
251     int sock_exit = socket(AF_INET, SOCK_DGRAM, 0); // socket temporal para enviar
252     EXIT
253
254     struct sockaddr_in dest_exit;
255     memset(&dest_exit, 0, sizeof(dest_exit)); // limpia la dirección destino
256     dest_exit.sin_family = AF_INET; // familia IPv4
257     dest_exit.sin_port = htons(PUERTO); // puerto del servidor
258     inet_aton(DIR_SERV, &dest_exit.sin_addr); // IP del servidor
259
260     Mensaje_C msj_exit;
261     memset(&msj_exit, 0, sizeof(Mensaje_C)); // limpia el mensaje de salida
262     strcpy(msj_exit.peticion, "EXIT"); // comando de apagado
263
264     // Envía el EXIT al servidor para que salga de su bucle
```

```
263     sendto(sock_exit, &msj_exit, sizeof(Mensaje_C), 0, (struct sockaddr *)&dest_exit
    , sizeof(dest_exit));
264
265     close(sock_exit); // libera el socket temporal
266     exit(0);         // el proceso hijo coordinador termina
267 }
268 }
```

3.3.5 Pseudocódigo

Pseudocódigo: sinc_cristian.c

Algoritmo Sincronizacion_Cristian

Constantes y Estructuras (header.h)

- 1: Definir $PUERTO \leftarrow 3000$
- 2: Definir $RTT_MAX_MS \leftarrow 600$
- 3: Definir $DIR_SERV \leftarrow "127.0.0.1"$
- 4: Estructura *tiempo*
- 5: $horas, minutos, segundos, milisegundos \in \mathbb{Z}$
- 6: FinEstructura
- 7: Estructura *Mensaje_C*
- 8: $id_proceso \in \mathbb{Z}$
- 9: $peticion \in \text{Alfabeto}[200]$
- 10: $tiempo_envio, tiempo_servidor, tiempo_respuesta \in tiempo$
- 11: FinEstructura

Funciones Auxiliares de Tiempo

- 1: Procedimiento CAPTURAR_TIEMPO(t)
- 2: Leer reloj físico del sistema (CLOCK_REALTIME)
- 3: Guardar hora actual en estructura t
- 4: Fin Procedimiento
- 5: Procedimiento CONVERSION_MS(t)
- 6: Salir Tiempo transformado a milisegundos totales ($int64$)
- 7: Fin Procedimiento

Lógica del Servidor Maestro

- 1: Procedimiento EJECUTAR_SERVIDOR_C
- 2: Definir $socket \leftarrow \text{socket}(AF_INET, SOCK_DGRAM)$ ▷ Protocolo UDP
- 3: $\text{bind}(socket, PUERTO)$

```

4:  Mientras Verdadero Hacer
5:       $msj \leftarrow \text{RecibirDatagrama}(socket, cliente\_ip)$ 
6:      Si CompararCadena( $msj.peticion$ , "EXIT") = 0 Entonces
7:          Romper ▷ Apaga el servidor
8:      Sino Si CompararCadena( $msj.peticion$ , "ACTUALIZAR") = 0 Entonces
9:           $\text{capturar\_tiempo}(msj.tiempo\_servidor)$  ▷ Captura  $T_s$ 
10:         Dormir aleatoriamente para simular latencia de procesamiento
11:         CopiarCadena( $msj.peticion$ , "OK")
12:          $\text{EnviarDatagrama}(socket, msj, cliente\_ip)$  ▷ Devuelve estampa
13:     Fin Si
14: Fin Mientras
15: Cerrar( $socket$ )
16: Fin Procedimiento

```

Lógica del Cliente

```

1: Procedimiento EJECUTAR_CLIENTE_C(indice)
2:     Definir  $socket \leftarrow \text{socket}(AF\_INET, SOCK\_DGRAM)$ 
3:     Repetir
4:         CopiarCadena( $msj.peticion$ , "ACTUALIZAR")
5:          $msj.id\_proceso \leftarrow \text{ObtenerPID}()$ 
6:          $\text{capturar\_tiempo}(msj.tiempo\_envio)$  ▷ Paso 1: Captura  $T_0$ 
7:          $\text{EnviarDatagrama}(socket, msj, DIR\_SERV)$ 
8:          $\text{RecibirDatagrama}(socket, msj)$  ▷ Paso 2: Recibe  $T_s$  dentro de  $msj$ 
9:          $\text{capturar\_tiempo}(msj.tiempo\_respuesta)$  ▷ Paso 3: Captura  $T_1$ 
10:         $t0 \leftarrow \text{conversion\_ms}(msj.tiempo\_envio)$ 
11:         $t1 \leftarrow \text{conversion\_ms}(msj.tiempo\_respuesta)$ 
12:         $rango \leftarrow t1 - t0$  ▷ Cálculo del RTT
13:    Hasta que  $rango \leq RTT\_MAX\_MS$  ▷ Filtro contra redes congestionadas
14:         $ts \leftarrow \text{conversion\_ms}(msj.tiempo\_servidor)$ 
15:         $latencia \leftarrow rango/2$ 
16:         $hora\_corregida \leftarrow ts + latencia$  ▷ Aplicación Fórmula de Cristian
17:        Reconstruir  $hora\_corregida$  a formato  $H : M : S : ms$ 
18:        Imprimir hora ajustada
19:        Cerrar( $socket$ )
20: Fin Procedimiento

```

3.3.6 Ejecución de Cristian.

```

1 [SERVIDOR] Reloj Maestro iniciado en el puerto 3000...
2
3 [SERVIDOR] Petición recibida de PID 37183. Hora despachada: [17:42:20:792]

```

```
4
5 [CLIENTE #07 | PID 37183] ---> T0 (Salida)      : [17:42:20:792]
6 [CLIENTE #07 | PID 37183] ---> T1 (Llegada)     : [17:42:20:882]
7 [CLIENTE #07 | PID 37183] ---> Ts (Servidor)    : [17:42:20:792]
8 [CLIENTE #07 | PID 37183] ---> RTT              : 90 ms (intento(s): 1)
9 [CLIENTE #07 | PID 37183] ---> Latencia Calc    : 45 ms
10 [CLIENTE #07 | PID 37183] ---> RELOJ AJUSTADO   : [17:42:20:837]
11
12 [SERVIDOR] Petición recibida de PID 37185. Hora despachada: [17:42:20:882]
13
14 [CLIENTE #09 | PID 37185] ---> T0 (Salida)      : [17:42:20:794]
15 [CLIENTE #09 | PID 37185] ---> T1 (Llegada)     : [17:42:20:913]
16 [CLIENTE #09 | PID 37185] ---> Ts (Servidor)    : [17:42:20:882]
17 [CLIENTE #09 | PID 37185] ---> RTT              : 119 ms (intento(s): 1)
18 [CLIENTE #09 | PID 37185] ---> Latencia Calc    : 59 ms
19 [CLIENTE #09 | PID 37185] ---> RELOJ AJUSTADO   : [17:42:20:941]
20
21 [SERVIDOR] Petición recibida de PID 37177. Hora despachada: [17:42:20:913]
22
23 [CLIENTE #01 | PID 37177] ---> T0 (Salida)      : [17:42:20:804]
24 [CLIENTE #01 | PID 37177] ---> T1 (Llegada)     : [17:42:21:106]
25 [CLIENTE #01 | PID 37177] ---> Ts (Servidor)    : [17:42:20:913]
26 [CLIENTE #01 | PID 37177] ---> RTT              : 302 ms (intento(s): 1)
27 [CLIENTE #01 | PID 37177] ---> Latencia Calc    : 151 ms
28 [CLIENTE #01 | PID 37177] ---> RELOJ AJUSTADO   : [17:42:21:64]
29
30 [SERVIDOR] Petición recibida de PID 37179. Hora despachada: [17:42:21:106]
31
32 [CLIENTE #03 | PID 37179] ---> T0 (Salida)      : [17:42:20:819]
33 [CLIENTE #03 | PID 37179] ---> T1 (Llegada)     : [17:42:21:243]
34 [CLIENTE #03 | PID 37179] ---> Ts (Servidor)    : [17:42:21:106]
35 [CLIENTE #03 | PID 37179] ---> RTT              : 424 ms (intento(s): 1)
36 [CLIENTE #03 | PID 37179] ---> Latencia Calc    : 212 ms
37 [CLIENTE #03 | PID 37179] ---> RELOJ AJUSTADO   : [17:42:21:318]
38
39 [SERVIDOR] Petición recibida de PID 37178. Hora despachada: [17:42:21:243]
40 [CLIENTE #02] RTT = 642 ms demasiado alto, reintentando... (intento 1)
41
42 [SERVIDOR] Petición recibida de PID 37184. Hora despachada: [17:42:21:491]
43 [CLIENTE #08] RTT = 654 ms demasiado alto, reintentando... (intento 1)
44
45 [SERVIDOR] Petición recibida de PID 37186. Hora despachada: [17:42:21:529]
46 [CLIENTE #10] RTT = 831 ms demasiado alto, reintentando... (intento 1)
```

```
47
48 [SERVIDOR] Petición recibida de PID 37181. Hora despachada: [17:42:21:715]
49 [CLIENTE #05] RTT = 889 ms demasiado alto, reintentando... (intento 1)
50
51 [SERVIDOR] Petición recibida de PID 37180. Hora despachada: [17:42:21:776]
52 [CLIENTE #04] RTT = 1001 ms demasiado alto, reintentando... (intento 1)
53
54 [SERVIDOR] Petición recibida de PID 37182. Hora despachada: [17:42:21:893]
55 [CLIENTE #06] RTT = 1128 ms demasiado alto, reintentando... (intento 1)
56
57 [SERVIDOR] Petición recibida de PID 37178. Hora despachada: [17:42:22:34]
58 [CLIENTE #02] RTT = 646 ms demasiado alto, reintentando... (intento 2)
59
60 [SERVIDOR] Petición recibida de PID 37184. Hora despachada: [17:42:22:137]
61 [CLIENTE #08] RTT = 797 ms demasiado alto, reintentando... (intento 2)
62
63 [SERVIDOR] Petición recibida de PID 37186. Hora despachada: [17:42:22:327]
64 [CLIENTE #10] RTT = 681 ms demasiado alto, reintentando... (intento 2)
65
66 [SERVIDOR] Petición recibida de PID 37181. Hora despachada: [17:42:22:396]
67 [CLIENTE #05] RTT = 640 ms demasiado alto, reintentando... (intento 2)
68
69 [SERVIDOR] Petición recibida de PID 37180. Hora despachada: [17:42:22:416]
70 [CLIENTE #04] RTT = 621 ms demasiado alto, reintentando... (intento 2)
71
72 [SERVIDOR] Petición recibida de PID 37182. Hora despachada: [17:42:22:514]
73
74 [CLIENTE #06 | PID 37182] ---> T0 (Salida)      : [17:42:22:34]
75 [CLIENTE #06 | PID 37182] ---> T1 (Llegada)     : [17:42:22:628]
76 [CLIENTE #06 | PID 37182] ---> Ts (Servidor)    : [17:42:22:514]
77 [CLIENTE #06 | PID 37182] ---> RTT              : 594 ms (intento(s): 2)
78 [CLIENTE #06 | PID 37182] ---> Latencia Calc   : 297 ms
79 [CLIENTE #06 | PID 37182] ---> RELOJ AJUSTADO   : [17:42:22:811]
80
81 [SERVIDOR] Petición recibida de PID 37178. Hora despachada: [17:42:22:628]
82 [CLIENTE #02] RTT = 672 ms demasiado alto, reintentando... (intento 3)
83
84 [SERVIDOR] Petición recibida de PID 37184. Hora despachada: [17:42:22:809]
85 [CLIENTE #08] RTT = 665 ms demasiado alto, reintentando... (intento 3)
86
87 [SERVIDOR] Petición recibida de PID 37186. Hora despachada: [17:42:22:992]
88 [CLIENTE #10] RTT = 786 ms demasiado alto, reintentando... (intento 3)
89
```

```
90 [SERVIDOR] Petición recibida de PID 37181. Hora despachada: [17:42:23:182]
91 [CLIENTE #05] RTT = 1022 ms demasiado alto, reintentando... (intento 3)
92
93 [SERVIDOR] Petición recibida de PID 37180. Hora despachada: [17:42:23:438]
94 [CLIENTE #04] RTT = 1129 ms demasiado alto, reintentando... (intento 3)
95
96 [SERVIDOR] Petición recibida de PID 37178. Hora despachada: [17:42:23:643]
97 [CLIENTE #02] RTT = 1030 ms demasiado alto, reintentando... (intento 4)
98
99 [SERVIDOR] Petición recibida de PID 37184. Hora despachada: [17:42:23:839]
100 [CLIENTE #08] RTT = 950 ms demasiado alto, reintentando... (intento 4)
101
102 [SERVIDOR] Petición recibida de PID 37186. Hora despachada: [17:42:23:942]
103 [CLIENTE #10] RTT = 916 ms demasiado alto, reintentando... (intento 4)
104
105 [SERVIDOR] Petición recibida de PID 37181. Hora despachada: [17:42:24:98]
106 [CLIENTE #05] RTT = 826 ms demasiado alto, reintentando... (intento 4)
107
108 [SERVIDOR] Petición recibida de PID 37180. Hora despachada: [17:42:24:264]
109 [CLIENTE #04] RTT = 643 ms demasiado alto, reintentando... (intento 4)
110
111 [SERVIDOR] Petición recibida de PID 37178. Hora despachada: [17:42:24:286]
112
113 [CLIENTE #02 | PID 37178] ---> T0 (Salida)      : [17:42:23:839]
114 [CLIENTE #02 | PID 37178] ---> T1 (Llegada)     : [17:42:24:309]
115 [CLIENTE #02 | PID 37178] ---> Ts (Servidor)    : [17:42:24:286]
116 [CLIENTE #02 | PID 37178] ---> RTT              : 470 ms (intento(s): 5)
117 [CLIENTE #02 | PID 37178] ---> Latencia Calc   : 235 ms
118 [CLIENTE #02 | PID 37178] ---> RELOJ AJUSTADO   : [17:42:24:521]
119
120 [SERVIDOR] Petición recibida de PID 37184. Hora despachada: [17:42:24:309]
121 [CLIENTE #08] RTT = 632 ms demasiado alto, reintentando... (intento 5)
122
123 [SERVIDOR] Petición recibida de PID 37186. Hora despachada: [17:42:24:574]
124 [CLIENTE #10] RTT = 651 ms demasiado alto, reintentando... (intento 5)
125
126 [SERVIDOR] Petición recibida de PID 37181. Hora despachada: [17:42:24:749]
127
128 [CLIENTE #05 | PID 37181] ---> T0 (Salida)      : [17:42:24:264]
129 [CLIENTE #05 | PID 37181] ---> T1 (Llegada)     : [17:42:24:852]
130 [CLIENTE #05 | PID 37181] ---> Ts (Servidor)    : [17:42:24:749]
131 [CLIENTE #05 | PID 37181] ---> RTT              : 588 ms (intento(s): 5)
132 [CLIENTE #05 | PID 37181] ---> Latencia Calc   : 294 ms
```

```
133 [CLIENTE #05 | PID 37181] ---> RELOJ AJUSTADO : [17:42:25:43]
134
135 [SERVIDOR] Petición recibida de PID 37180. Hora despachada: [17:42:24:852]
136 [CLIENTE #04] RTT = 780 ms demasiado alto, reintentando... (intento 5)
137
138 [SERVIDOR] Petición recibida de PID 37184. Hora despachada: [17:42:25:66]
139 [CLIENTE #08] RTT = 672 ms demasiado alto, reintentando... (intento 6)
140
141 [SERVIDOR] Petición recibida de PID 37186. Hora despachada: [17:42:25:246]
142 [CLIENTE #10] RTT = 631 ms demasiado alto, reintentando... (intento 6)
143
144 [SERVIDOR] Petición recibida de PID 37180. Hora despachada: [17:42:25:380]
145
146 [CLIENTE #04 | PID 37180] ---> T0 (Salida)      : [17:42:25:66]
147 [CLIENTE #04 | PID 37180] ---> T1 (Llegada)     : [17:42:25:403]
148 [CLIENTE #04 | PID 37180] ---> Ts (Servidor)    : [17:42:25:380]
149 [CLIENTE #04 | PID 37180] ---> RTT              : 337 ms (intento(s): 6)
150 [CLIENTE #04 | PID 37180] ---> Latencia Calc    : 168 ms
151 [CLIENTE #04 | PID 37180] ---> RELOJ AJUSTADO   : [17:42:25:548]
152
153 [SERVIDOR] Petición recibida de PID 37184. Hora despachada: [17:42:25:403]
154
155 [CLIENTE #08 | PID 37184] ---> T0 (Salida)      : [17:42:25:246]
156 [CLIENTE #08 | PID 37184] ---> T1 (Llegada)     : [17:42:25:637]
157 [CLIENTE #08 | PID 37184] ---> Ts (Servidor)    : [17:42:25:403]
158 [CLIENTE #08 | PID 37184] ---> RTT              : 391 ms (intento(s): 7)
159 [CLIENTE #08 | PID 37184] ---> Latencia Calc    : 195 ms
160 [CLIENTE #08 | PID 37184] ---> RELOJ AJUSTADO   : [17:42:25:598]
161
162 [SERVIDOR] Petición recibida de PID 37186. Hora despachada: [17:42:25:637]
163
164 [CLIENTE #10 | PID 37186] ---> T0 (Salida)      : [17:42:25:380]
165 [CLIENTE #10 | PID 37186] ---> T1 (Llegada)     : [17:42:25:935]
166 [CLIENTE #10 | PID 37186] ---> Ts (Servidor)    : [17:42:25:637]
167 [CLIENTE #10 | PID 37186] ---> RTT              : 555 ms (intento(s): 7)
168 [CLIENTE #10 | PID 37186] ---> Latencia Calc    : 277 ms
169 [CLIENTE #10 | PID 37186] ---> RELOJ AJUSTADO   : [17:42:25:914]
170
171 [SERVIDOR] Comando EXIT recibido. Apagando.
172
173 [SISTEMA] Simulación finalizada.
```

Conclusiones

El desarrollo de esta práctica resultó un verdadero reto, principalmente por la necesidad de establecer conexiones P2P y Cliente-Servidor para gestionar distintos algoritmos de sincronización de manera simultánea. Al final, se demostró que estos algoritmos pueden integrarse exitosamente en múltiples procesos, adaptándose a las necesidades de diferentes arquitecturas de red.

Este proyecto me permitió observar de primera mano cómo operan las aplicaciones a bajo nivel. Demostró la importancia vital de mantener un control absoluto sobre los procesos creados (**forks** e hilos), garantizando su correcta sincronización para evitar problemas lógicos, de concurrencia o de latencia. Se logró implementar una comunicación efectiva mediante sockets TCP y UDP; sin embargo, esto representa un desafío constante para el programador. Al ser la base sobre la cual opera toda aplicación de red, su complejidad exige un diseño sumamente estructurado. Quedó claro que es imposible comprender, modificar o escalar un sistema distribuido si no se cuenta con un control de ejecución riguroso y una documentación estricta sobre el ciclo de vida de cada proceso.

Referencias

- CRUD - Glosario de MDN Web Docs — MDN. (s. f.). <https://developer.mozilla.org/es/docs/Glossary/CRUD>
- Amazon Web Services. (s. f.). ¿Qué es un cliente ligero? Amazon Web Services, Inc. <https://aws.amazon.com/es/what-is/thin-client/>
- Amazon Web Services. (s. f.-b). Qué es una CLI (interfaz de la línea de comandos)? Amazon Web Services, Inc. Amazon Web Services. (s. f.-b). Qué es una CLI (interfaz de la línea de comandos)? Amazon Web Services, Inc. <https://aws.amazon.com/es/what-is/cli>